

Aurviz

A personal AI agent for Android

Project Report

Requirements · Architecture · Testing · Performance · Guides

13 November 2026

Contents

1. Software Requirements Specification
2. Architecture
3. Test Report
4. Performance
5. Evaluator Guide
6. Email Flow with the Test Account
7. The Aurviz Browser
8. User Guide
9. Backend API
10. Build and Deployment

Software Requirements Specification

Product: Aurviz, a personal AI agent for Android **Version:** 1.1 (November 2026) **Status:** Implemented and verified (see Test Report)

1. Introduction

1.1 Purpose

This document specifies what Aurviz must do (functional requirements) and how well it must do it (non-functional requirements). Every requirement has an ID, an acceptance criterion and a test, so that a reviewer can verify it directly.

1.2 Scope

Aurviz is an Android application, backed by a small Node.js service, that understands spoken and typed instructions, remembers the user's preferences, plans multi-step work, calls tools (Gmail, memory, scheduler, web browser) and executes tasks, including scheduled tasks that run when the app is closed. It is more than a chat bot: the model decides *what* to do, Kotlin code *does* it, and WorkManager decides *when*.

1.3 Definitions

Term	Meaning
Agent	The orchestration loop that asks Gemini for the next step, runs the tool it picks, and feeds the result back
Tool	A Kotlin class implementing `AgentTool` that performs one operation (for example `search_emails`)
Sensitive tool	A tool with an external side effect (sending mail, clicking in a web page). It always needs explicit user approval
Demo mailbox	A built-in sample inbox that lets the app be evaluated without any Google account
HITL	Human in the loop: the approval dialog shown before a sensitive action

1.4 Responsibilities (three separate concerns)

Concern	Owner
Reasoning: decide what to do next	Gemini (function calling)
Execution: perform the operation	Kotlin tools and the backend
Scheduling: run later	Android WorkManager

Gemini cannot wake the app. A scheduled task is persisted in Room and registered with WorkManager, which later invokes the same agent loop in background mode.

2. Overall description

2.1 Users

- **End user:** a person who wants an assistant for email, reminders and quick web tasks.
- **Evaluator:** a reviewer who must be able to test every feature without receiving anyone's credentials (see Evaluator Guide).

2.2 Operating environment

Android 8.0 (API 26) or newer, internet access, a Gemini API key (bundled at build time or entered in Settings). Mail features work with the shared test inbox, the demo mailbox or the user's own Google account.

2.3 Constraints

- WorkManager periodic work is durable but not exact to the minute (Doze and battery policy apply).
- Google restricts the `gmail.readonly` scope: accounts must be listed as OAuth test users until the app passes Google verification.
- The controlled browser only opens public HTTPS pages.

2.4 Assumptions

The backend is reachable over HTTPS. The Gemini free tier may rate-limit requests; the app handles this (retry, model fallback).

3. Functional requirements

Priority: **P0** must have, **P1** should have, **P2** nice to have.

3.1 Core requirements

ID	Requirement	Pri	Acceptance criterion	Test
FR-01	**Voice input.** Accept spoken commands and transcribe them with Android speech recognition	P0	Tapping the microphone, speaking, and the transcribed text appears as a "Voice" message and is processed	Device (earlier build); `VoiceManager`
FR-02	**Text input.** Accept typed commands	P0	A typed prompt receives an answer	`AgentOrchestratorTest` ; device
FR-03	**Intent detection.** Recognise what the user wants and which tool is needed	P0	"Check my emails" triggers `search_emails` ; "draft, don't send" triggers `create_email_draft` and never `send_email`	`AgentReliabilityTest` ; device
FR-04	**Tool calling.** Invoke tools with structured arguments and return results to the model	P0	The timeline shows each tool call with status and result	`AgentOrchestratorTest` , `GeminiAgentClientTest` ; device
FR-05	**Persistent memory.** Save facts and preferences that survive restarts	P0	Save a memory, force-stop the app, reopen, the memory is still listed	Device; `MemoryToolTest`
FR-06	**Memory retrieval.** Use saved memories automatically	P0	After "I prefer formal emails", a later draft is formal	`AgentReliabilityTest.savedPreference…` ; device
FR-07	**Memory edit and delete.** View, edit and delete memories	P0	Edit and delete work from the Memory screen (with confirmation) and by voice ("forget my…")	`MemoryToolTest` ; device
FR-08	**Email sending with confirmation.** Send only after the user approves recipient, subject and body	P0	An editable approval dialog appears; cancelling sends nothing; approving sends once	`AgentOrchestratorTest.processUserPrompt_sensitiveTool…` ; device
FR-09	**Web automation.** Open pages, read, scroll, navigate and extract information	P1	"Open news.ycombinator.com and tell me the top 3 stories" opens the site and lists stories	`ControlledWebScraperTest` , `BrowserUrlTest` ; device
FR-10	**Task scheduling.** Create one-time and recurring tasks from natural language or cron-like rules (daily, weekdays, weekends, specific days, 5-field cron)	P0	"Every morning at 9 AM…" shows "Daily at 09:00"; "Every weekday at 5:14 PM" shows "Weekdays at 17:14" and runs at that time; the next run skips the weekend	`CronScheduleTest` , `SchedulerToolTest` , `TimeUtilsTest` ; device
FR-11	**Background execution.** Run tasks without the app open	P0	A task run through WorkManager records a result and posts a notification	`TaskExecutionPolicyTest` ; device
FR-12	**Multi-step execution.** Complete several operations in sequence	P0	"Check emails, find interview invitations and summarize them" runs search, then read, then summary	`AgentReliabilityTest` ; device
FR-13	**Task history.** Show previous runs	P0	The History tab lists each run with status, time, duration and attempt	Device
FR-14	**Error reporting.** Show useful errors and recovery actions	P0	Failures show a plain message and a "Try again" action for transient errors	`AgentReliabilityTest` , `ExecutionLogFormatterTest` ; device

3.2 Extended requirements (beyond the brief)

ID	Requirement	Pri	Acceptance criterion	Test
FR-15	**Email search and read.** Search a mailbox with Gmail operators and read a message	P0	`subject:` , `from:` , `is:unread` , `newer_than:` filters work	`DemoMailServiceTest` , `gmail-inbox.test.js`
FR-16	**Email drafts.** Save a draft without sending	P0	The draft is stored (Gmail Drafts or demo Drafts) and nothing is sent	`GmailInboxToolsTest` ; device
FR-17	**Demo mailbox.** Test every email feature without an account	P0	Turning on Demo mailbox lets search, read, draft and send work on sample data; nothing leaves the device	`DemoMailServiceTest` ; device
FR-18	**OAuth Gmail connection.** Connect, check and disconnect a Gmail account via OAuth	P0	No password is ever entered in the app; disconnect revokes access	Device
FR-19	**Task management.** Pause, resume, run now, edit and delete tasks	P1	A paused task does not run; "Run now" runs immediately	`TaskExecutionPolicyTest` ; device
FR-20	**Readable run results.** Describe task outcomes in plain language with a next step	P1	A failed run explains the cause and what to do; raw text sits behind "Technical details"	`ExecutionLogFormatterTest`
FR-21	**Browser tools for people.** Address bar with search, find in page, reader view, desktop site, text size, bookmarks, history, share	P1	Each control works in the in-app browser	`BrowserUrlTest` ; device
FR-22	**AI on web pages.** Summarize, extract key facts, explain or ask about the open page	P1	Tapping Summarize returns to chat with a summary of that page	Device

ID	Requirement	Pri	Acceptance criterion	Test
FR-23	**Theme.** Choose System, Light or Dark	P1	The choice applies immediately and persists	Device
FR-24	**Onboarding and shortcuts.** First-run walkthrough, launcher shortcuts, share text to Aurviz	P2	Onboarding appears once; long-pressing the icon lists shortcuts; sharing text opens a summary	Device
FR-26	**Test Gmail account for reviewers.** The app shows the test account's email, password and numbered sign-in steps	P0	A reviewer follows the steps, signs in on Google's page and sees "Connected as…"; search, read and draft work; mail can only be sent to the account itself or example.com addresses; it cannot be disconnected	`shared-account.test.js` ; device
FR-27	**Demo mailbox viewer.** See the demo mailbox's Inbox, Sent and Drafts, read messages, and watch them update	P0	A draft or sent message saved by the agent appears without leaving the screen; demo can be reset. Real accounts are viewed in Gmail	`MailGatewayTest` ; device
FR-28	**Mailbox selection.** Choose Test Gmail account, My own Gmail or Demo mailbox at any time	P0	The selection persists, the header badge reflects it, and the agent uses the selected mailbox immediately	`MailGatewayTest` ; device
FR-25	**Settings and permissions.** Manage key, default recipient, spoken replies, Gmail and system permissions	P1	All settings persist across restarts	Device

4. Non-functional requirements

ID	Category	Requirement	Measure and result
NFR-01	Performance	Simple commands respond within about 3 s, excluding external service time	On-device overhead about 0.1 s median; end-to-end median 1.6 s over 30 samples (Performance)
NFR-02	Security	Protect keys, tokens and accounts	OAuth only (no passwords); refresh tokens AES-256-GCM encrypted on the server; session token and API key in Keystore-backed encrypted storage; excluded from backup; confirmation before send; no secrets in git
NFR-03	Reliability	Failures never look like successes; state survives failures	Failed runs are recorded as failed; transient errors retry with exponential backoff (up to 3); ambiguous email delivery is stored as UNKNOWN and never auto-resent
NFR-04	Accuracy	Choose the correct tool and clarify ambiguity	Draft-versus-send rule; asks for a missing recipient address; never invents email content
NFR-05	Maintainability	Modular, easy to extend	Layered packages; new tool needs a class, a declaration and one registration line (Architecture)
NFR-06	Scalability	Support more tools and users without redesign	`AgentTool` and `MailService` abstractions; backend is stateless with a PostgreSQL job queue
NFR-07	Fault tolerance	Handle outages gracefully	Offline check before background runs; retry actions; model fallback chain; backend cold-start-safe timeouts and warm-up
NFR-08	Usability	Intuitive interface	One-tap mic, live "what the agent is doing" status, tool timeline, approval dialogs, onboarding, dark mode
NFR-09	Resource efficiency	Low battery and memory impact	WorkManager only (no persistent service), network constraints, bounded tables and context
NFR-10	Compatibility and responsiveness	Wide device support and any screen size	minSdk 26, adaptive and themed icon, runtime permissions, tested on Android 16. Layouts adapt: scrolling pages on short screens, one-line titles and labels under large fonts (checked at font scale 1.6), and a centered 640 dp column on tablets. Checked at 360 x 640 dp, 411 x 914 dp and 600 x 960 dp
NFR-12	Safe shared access	A reviewer-facing account must not be abusable	Server-enforced: sends only to itself or example.com addresses, no disconnect; throwaway account
NFR-11	Privacy	Minimise data exposure	Page and text attachments are sent only when the user triggers an action; demo mode never contacts Gmail

5. External interfaces

Interface	Direction	Purpose
Gemini REST `generateContent`	App to Google	Reasoning and function calling, with model fallback
Aurviz backend REST (HTTPS)	App to server	OAuth session, Gmail search/read/draft/send, scheduled email
Gmail API	Server to Google	Mailbox operations with the user's consent
Android SpeechRecognizer / TextToSpeech	App to OS	Voice in and out
Android WorkManager / Notifications	App to OS	Background execution and result alerts

Backend endpoints are listed in API.

6. Data requirements

On device (Room): `memories`, `scheduled_tasks`, `task_executions`, `performance_samples`. Chat history is a bounded JSON file (100 messages). Bookmarks and history for the browser are stored locally. **On the server (PostgreSQL):** `GoogleAccount` (encrypted refresh token), `AppAuthCode`, `DeviceSession` (hashed tokens), `ScheduledEmail`, `EmailExecution`, plus the pg-boss job tables.

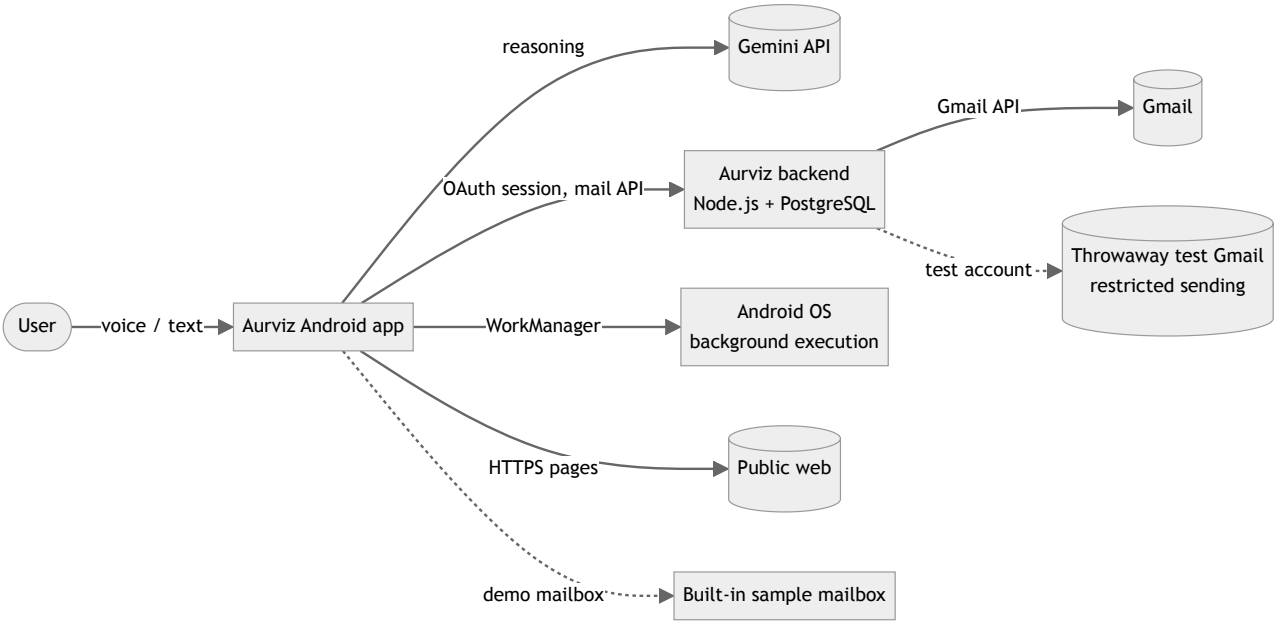
7. Traceability

See the matrix in Test Report.

Architecture

Aurviz is an Android app (Kotlin, Jetpack Compose) plus a small Node.js backend. This document explains how the pieces fit together, why they are separated the way they are, and how to extend the system.

1. System context



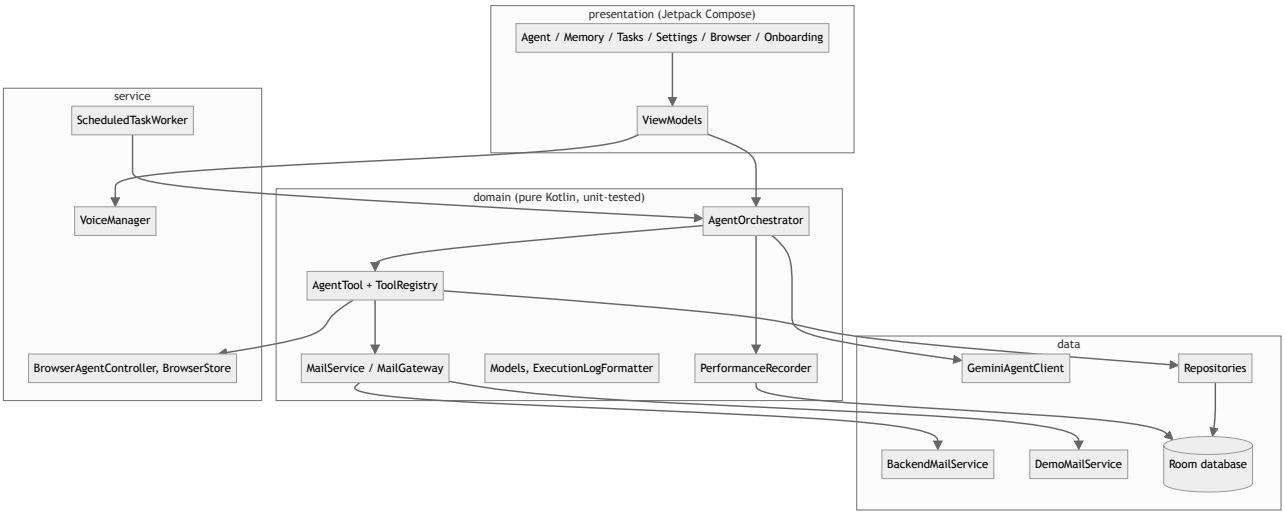
2. Three separate concerns

The most important design decision: **thinking, doing and timing are three different things owned by three different components.**

Concern	Owner	Where in code
Reasoning: decide the next step	Gemini via function calling	`GeminiAgentClient`, prompt in `AgentOrchestrator`
Execution: perform the operation	Kotlin `AgentTool`s and the backend	`data/tools/*`, `backend/src/routes/*`
Scheduling: run later	Android WorkManager (persists its own state)	`TaskSchedulerRepositoryImpl`, `ScheduledTaskWorker`

Gemini cannot wake an app. A scheduled task is stored in Room and registered with WorkManager. When WorkManager runs the worker, the worker calls the same agent loop in **background mode**, which has stricter safety rules (see section 6).

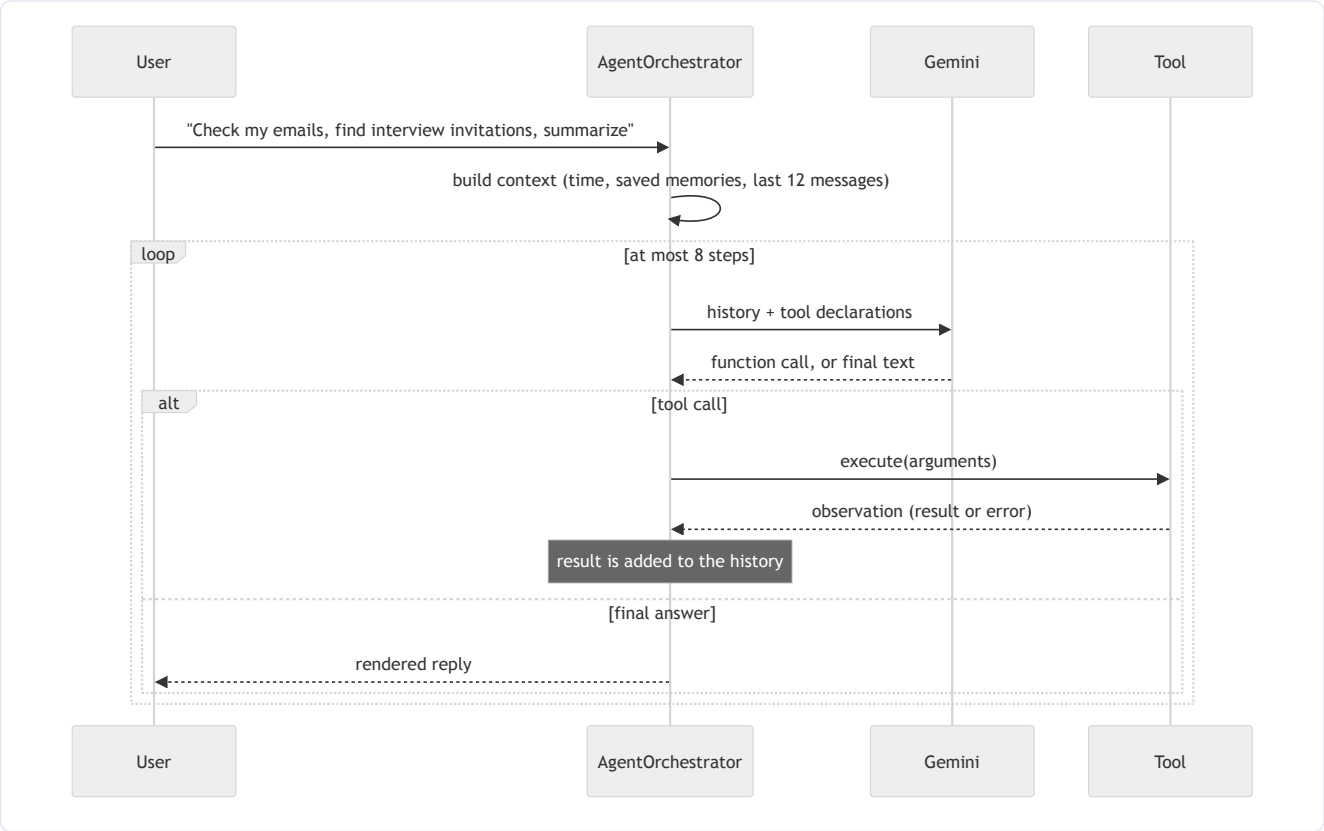
3. Layers of the Android app



Rules that keep it maintainable:

- `domain` never imports Android UI classes, so the orchestrator is covered by plain JVM tests with fake clients.
- Tools depend on interfaces (`MailService`, repositories), not on concrete networking classes.
- The UI observes `StateFlow`s and holds no business logic.

4. The agent loop

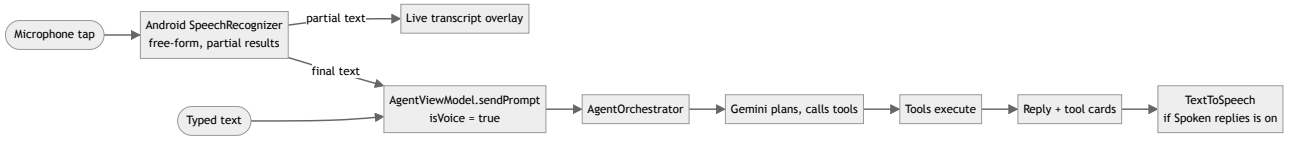


Behaviours worth knowing:

- **Planning rules** are in the system instruction: one tool at a time, search then read then summarize, draft versus send, ask when the recipient is ambiguous, never invent mail content.
- **Memory** (lexical match plus the most recent entries) is injected on every turn, so a saved preference like "formal emails" affects later drafts.
- **Step limit:** a request cannot loop forever. Reaching the limit shows an explicit message with a retry action.
- **Empty answers:** if the model returns no real text (for example a lone dash) the orchestrator asks once more, then falls back to a retryable error.
- **Attachments:** the browser and share-intent can pass page or text content to the model. It is sent only for the action the user triggered and is marked as untrusted data, which reduces prompt-injection risk.

4.1 Voice-to-action pipeline

Voice and text share one path. Speech is converted to text on the device, then the same agent loop runs.

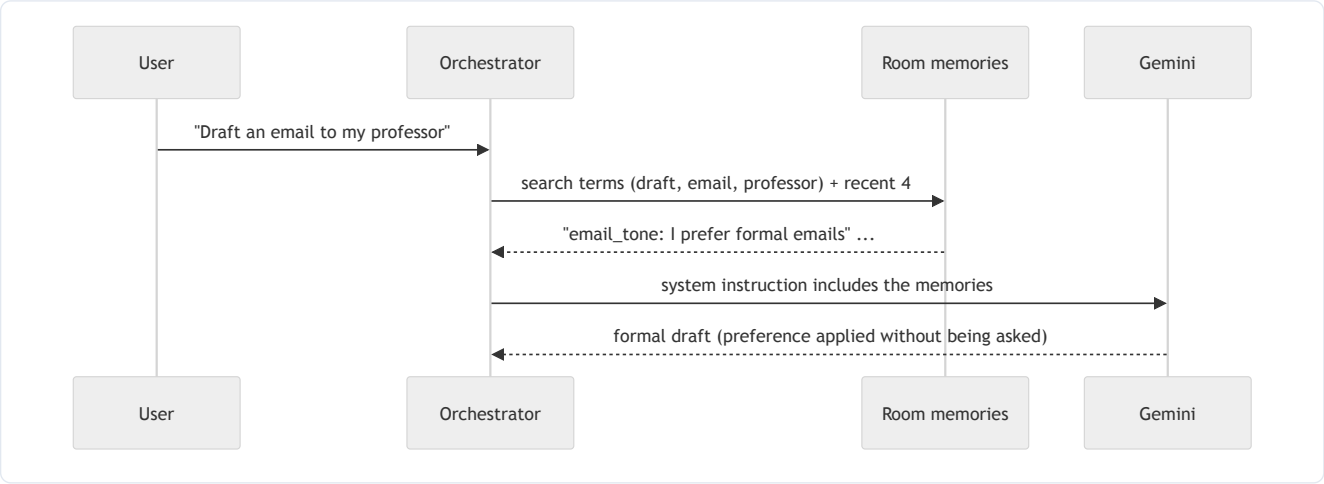


1. **Capture:** `VoiceManager` starts `SpeechRecognizer` with `LANGUAGE_MODEL_FREE_FORM` and partial results. The microphone permission is requested at launch, and a missing recognizer is handled without crashing.
2. **Feedback:** partial results and the audio level drive an animated overlay, so the user sees what was heard.
3. **Hand-off:** the final transcript is sent with `isVoiceInput = true`, so the chat labels the message "Voice". From here voice and typing are identical.
4. **Plan and act:** the orchestrator runs the reason, act, observe loop (section 4) and shows each tool as a card.
5. **Report:** the answer is rendered, and spoken aloud through `TextToSpeech` when "Spoken replies" is enabled. A stop button interrupts speech.

Speech recognition is on-device or platform-provided; only the resulting text goes to Gemini, never audio.

4.2 Memory storage and retrieval

Aspect	Design
Storage	Room table `memories` (key, content, category, importance, createdAt). Survives restarts and app updates through an explicit migration
Writing	The agent's `manage_memory` tool (save, update, delete) or the Memory screen (add, edit, delete with confirmation). Saving an existing key updates it instead of duplicating
Retrieval	`AgentOrchestrator.memoryContextFor(prompt)` : tokenize the prompt, drop stop-words, run a SQL `LIKE` search per term across key, content and category (ordered by importance, then recency), then add the 4 most recent memories so general preferences always apply
Injection	The top results (at most 8) go into the system instruction as "Known user context and saved preferences (apply them)" on **every** turn, including background runs
Safety	Update and delete act only on one unambiguous match; otherwise the agent receives the candidates and must ask the user



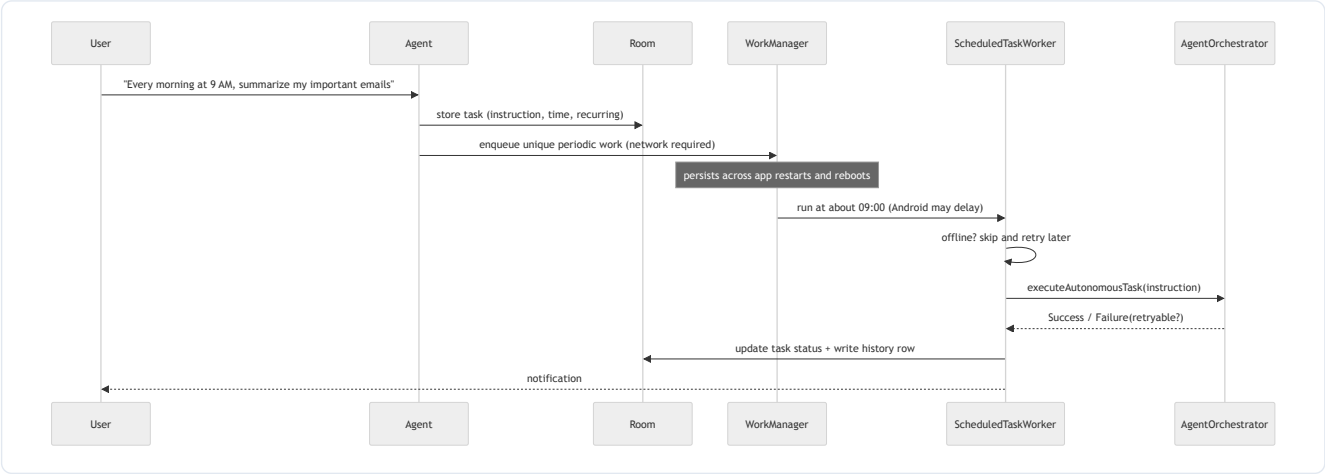
Why lexical search: it is fast (about 54 ms median measured), private (nothing leaves the phone for retrieval) and needs no model download. Its limit is that it matches words, not meaning, so a vector index is the natural upgrade.

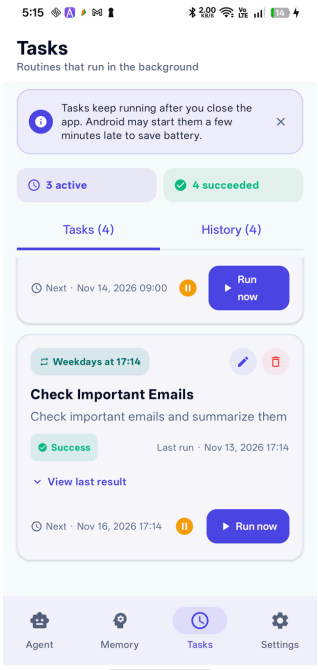
5. Tools and the approval gate

Tool	Side effect	Needs approval
`search_emails` , `read_email`	none	no
`create_email_draft`	saves a draft, sends nothing	no
`send_email`	sends mail	**yes**
`manage_memory` (save, recall, update, delete)	local data	no
`schedule_task`	registers a background job	no
`browse_web` , `open_url` , `read_page` , `scroll_page` , `extract_links` , `navigate_*`	none	no
`click_element` , `enter_text`	can change a web page	**yes**

A tool marked `isSensitive` pauses the loop and shows an editable confirmation dialog. Only approval calls `executeConfirmed` . In background runs a sensitive tool is never executed at all; the model is told it was blocked and returns a draft or summary instead.

6. Scheduled tasks





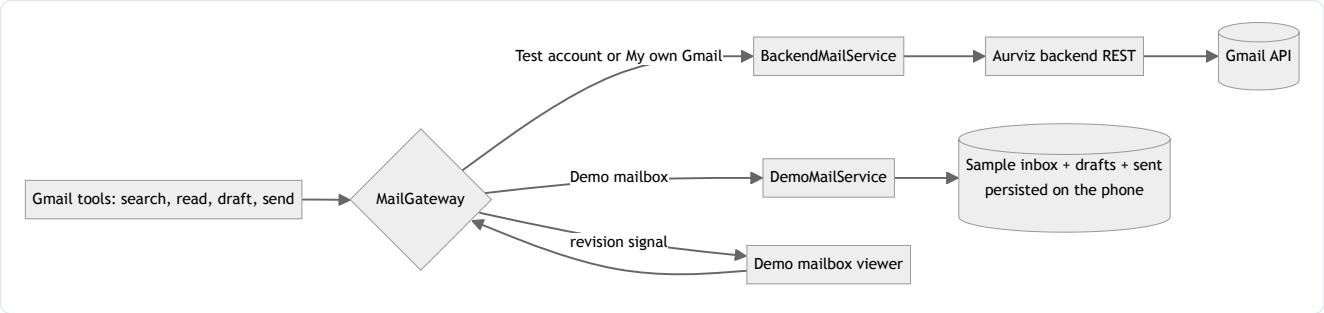
A weekday task after it ran at 17:14: next run is Monday

Reliability rules:

- **Cron-like scheduling.** The model turns a sentence into a `recurrence` and a time: `daily`, `weekdays`, `weekends`, day names (`MON,THU`), `once`, or a full 5-field cron expression (`0 9 * * 1-5`). `CronSchedule` (pure Kotlin) parses the expression, supports `*`, lists, ranges, steps and `JAN-DEC` / `SUN-SAT` names, and computes the next run time (classic cron rule: if day-of-month and day-of-week are both set, either may match). The expression is stored with the task and shown as plain words, for example "Weekdays at 09:00".
- **A chain of exact one-time jobs, not a 24-hour periodic job.** A periodic WorkManager job can only repeat at a fixed interval, so it cannot express "weekdays". Instead each recurring task keeps one delayed one-time job. When a run finishes (not retrying), the worker computes the next occurrence from the cron expression and queues it (`APPEND_OR_REPLACE`, so it never cancels itself). At app start `ensureScheduled()` re-queues any task whose chain is missing and migrates tasks created by older builds (`daily_at_HH:MM`). Manual "Run now" never moves the schedule, and a paused task does not queue a next run.
- Android may still start a job a few minutes late under battery optimization, and a run missed while the phone was off is not replayed; the next occurrence is scheduled instead.
- Unique work names (`task_<id>`, `task_<id>_manual`) stop duplicate enqueues; an in-process guard drops a concurrent second run.
- A run is **never reported as success** if a required tool failed. Transient failures (offline, rate limit, tool outage) are retried with exponential backoff up to 3 attempts and shown as "Retrying".
- Pause cancels the WorkManager job but keeps the task; resume re-registers it. A run that was already in flight never silently resumes a paused task.
- Every attempt is stored in `task_executions` and shown in Tasks, History. Raw errors are translated by `ExecutionLogFormatter` into a plain explanation with a next step.

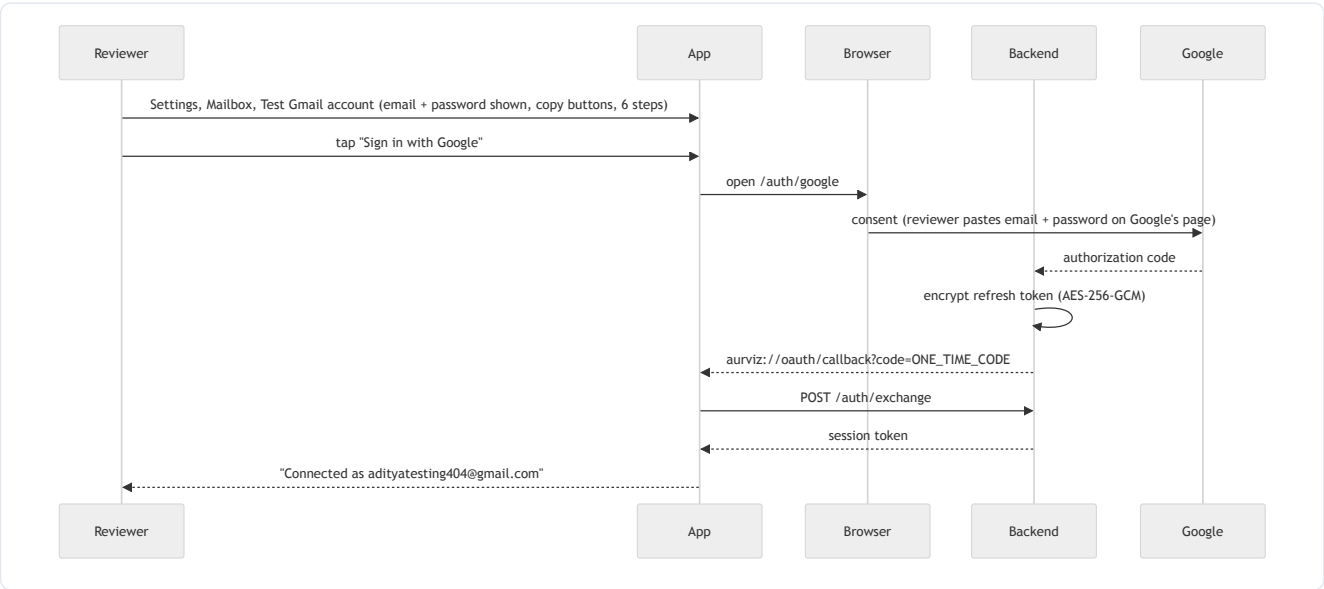
7. Mail: three mailboxes, one interface

The Gmail tools only know the `MailService` interface. A `MailGateway` chooses the implementation on every call, so switching mailbox takes effect instantly.



Source	Where the mail lives	Credentials	Notes
Test Gmail account (default)	A throwaway Gmail account	Email and password are shown in the app; the reviewer signs in on Google's page	Sends only to itself or example.com addresses; cannot be disconnected
My own Gmail	The user's Google account	Google sign-in (OAuth)	Account must be an OAuth test user until the app is verified
Demo mailbox	App-private storage on the phone	None	Ten sample emails; drafts and sent mail persist; **Reset** restores the sample

Reviewer sign-in flow (test account)

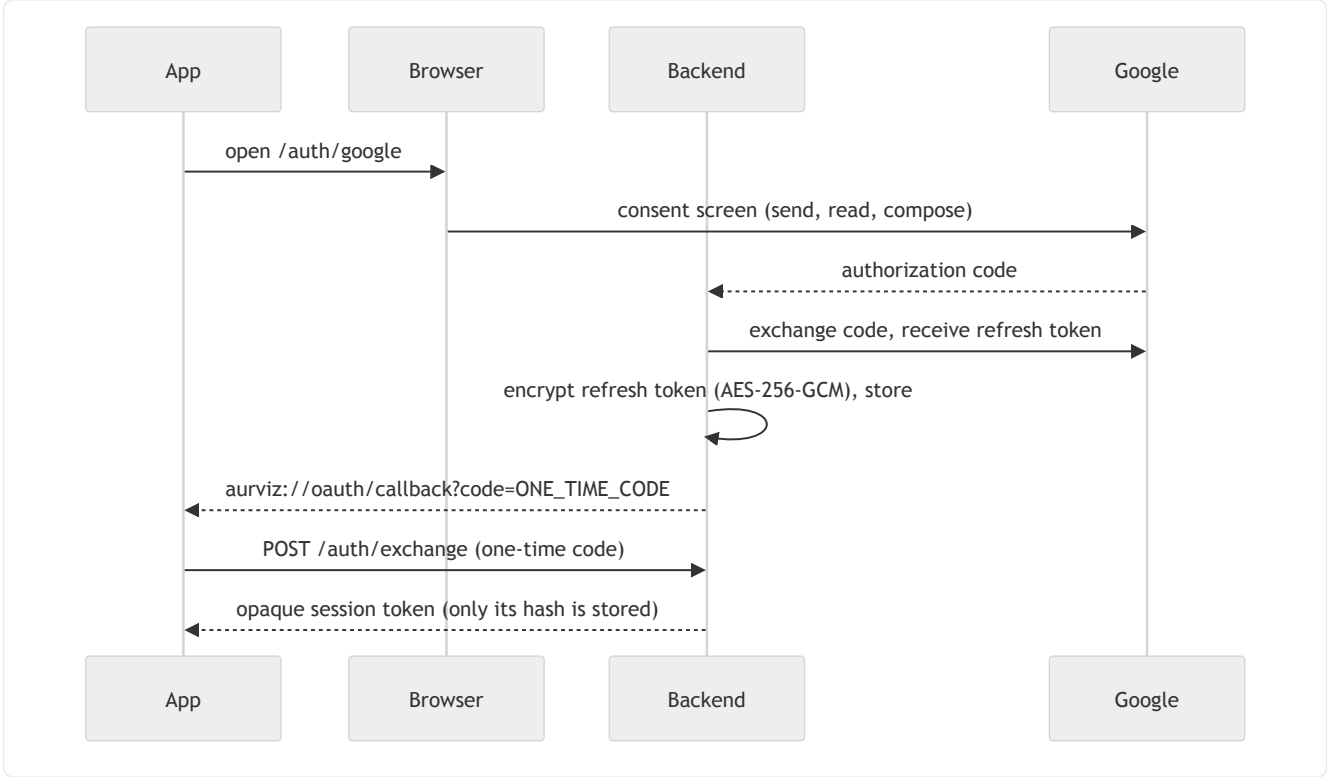


The credentials are typed only on Google's own page, never into Aurviz. The email is public; the password is injected at build time from the git-ignored `local.properties`, so it is shown in the APK but never committed to the repository. The server enforces the safeguards, so they cannot be bypassed by a modified app: `POST /emails/send` and `/emails/scheduled` reject any recipient other than the test account itself or a reserved example address (`example.com`, `.org`, `.net`, `*.example`, `*.test`) with 403, and `DELETE /auth/google` is refused for that account.

The demo mailbox viewer

One screen shows the demo mailbox with **Inbox, Sent and Drafts** tabs. When Aurviz saves a draft or sends mail, the gateway raises a `revision` signal and the viewer reloads, so the screen **updates live**. Real accounts are viewed in Gmail itself. Opening a message offers "Summarize with Aurviz", which passes the message to the agent as untrusted context.

Real accounts use OAuth through the backend:



The app never sees a Google password or refresh token. Because the refresh token lives on the server, scheduled email summaries work while the phone is idle.

8. Web browser

`BrowserAgentController` is a single shared session between the agent's browser tools and the visible WebView. People get an address bar that searches when you type words, find in page, reader view, desktop site, text size, bookmarks, history and share. Four AI actions (Summarize, Key facts, Explain simply, Ask) read the page text and hand it to the agent.

Safety: HTTPS only; private, local and link-local addresses are rejected (DNS is resolved for agent opens); no JavaScript bridge to the app; file and content access disabled; Safe Browsing on; clicks and typing by the agent need approval and never submit forms. This is a controlled WebView, not a hardened sandbox.

9. Data

Store	Technology	Content
`memories`	Room	key, content, category, timestamp
`scheduled_tasks`	Room	instruction, schedule, enabled, last status
`task_executions`	Room	per-run status, message, duration, attempt (500 rows kept)
`performance_samples`	Room	latency samples (1000 rows kept)
Chat	JSON file	last 100 messages
Secrets	EncryptedSharedPreferences	Gemini key, session token
Preferences	SharedPreferences	theme, mailbox source, onboarding
Demo mailbox	SharedPreferences (JSON)	user-created drafts and sent mail
Server	PostgreSQL + Prisma, pg-boss	accounts (encrypted token), sessions (hashed), scheduled emails, history, job queue

Database upgrades use explicit migrations (`MIGRATION_1_2`), so memories and tasks are never wiped on update.

10. Security

Threat	Mitigation
Stolen credentials	No passwords; OAuth with server-held refresh token (AES-256-GCM)
Token on device	Keystore-backed encrypted preferences, excluded from backup
Unwanted sending	Approval dialog for every send; background runs cannot send

Threat	Mitigation
Abuse of the shared test account	Server-enforced: sends only to itself or example addresses, no disconnect, throwaway account
Header injection in mail	Recipient and subject reject line breaks (app and server)
Replay or double send	Idempotency key per request; unique database constraint
Abuse of the API	Rate limiting, Helmet headers, 32 KB body limit, schema validation
Malicious web content	HTTPS only, private-address blocking, no JS bridge, content treated as untrusted data
Secrets in source	None committed; build-time Gemini key is read from the git-ignored `local.properties`

Known limit: a Gemini key bundled into an APK can be extracted from the file. For production, route Gemini through the backend with quotas.

11. Extending Aurviz

Adding a capability, for example Google Calendar, needs **no change to the orchestrator**:

1. Implement `AgentTool` (and a backend route if it needs Google credentials).
2. Add its function declaration in `GeminiAgentClient`.
3. Register it in `AurvizApp`.
4. Set `isSensitive = true` if it has external side effects: it then automatically gets the approval gate and is blocked in background runs.

Swapping the mail provider means implementing `MailService` once; the tools and orchestrator are untouched, which is exactly how the demo mailbox was added.

12. Decisions and trade-offs

Decision	Reason	Trade-off
Backend holds Google tokens	Scheduled mail works while the app is closed; no tokens on the phone	Needs a hosted service; free tier can sleep (the app warms it up and allows a long first request)
Gemini function calling	Reliable structured tool use	Depends on the model API; fallback chain mitigates model retirement
WorkManager for scheduling	Persistent, battery-aware	Not exact to the minute
Lexical memory search	Simple, fast, private	Not semantic; a vector index is a possible upgrade
Demo mailbox	Lets anyone evaluate without credentials	Sample data only
Test Gmail account	Reviewers use real Gmail with credentials shown in the app	Google may challenge a shared account; publish the consent screen so sign-in is not blocked, and keep the demo mailbox as a fallback

Test Report

Build under test: Aurviz 1.1 debug build, commit history on `main` **Date:** 13 November 2026 **Device:** realme RMX5116, Android 16 (API 36), Wi-Fi
Backend: deployed instance (`/health` OK), Gmail test account in OAuth testing mode

1. Summary

Area	Result
Android unit tests	**99 of 99 pass**
Backend tests	**18 of 18 pass** (Node 22)
Device checks	**All listed in section 3 pass**
Build	`assembleDebug` succeeds; CI runs tests and the build on every push
Dependency audit	`npm audit` reports no known vulnerabilities

Not verified on the device (stated openly): in the browser, the find-in-page match counter, desktop-site toggle, text-size and share actions (the menu entries are shown but typing into the find field could not be driven over adb); speech input in the latest build; offline Retry behaviour (unit-tested only, because switching Wi-Fi off would disconnect the wireless debugging link); an overnight run with the screen off for hours (a scheduled run was verified firing on time with the phone awake and charging); and the relaxed recipient rule (example.com addresses) on the deployed server, which needs the latest backend version deployed; its logic is unit-tested.

2. Automated tests

2.1 Android (JVM, `./gradlew app:testDebugUnitTest`)

Suite	Tests	What it proves
`AgentOrchestratorTest`	8	Direct answers, conversation persistence, function-call id round-trip, multi-step background runs, sensitive tools never run in the background, approval flow with edited fields
`AgentReliabilityTest`	10	Real errors with retry (no generic greeting), step limit never silent, multi-step email workflow, preference injected into the prompt, transient tool failure is retryable, latency recorded, empty model answer retried then reported, background runs never use screen-bound tools
`GeminiAgentClientTest`	11	Request building, model fallback, error classification, response parsing, thought-signature preservation
`TaskExecutionPolicyTest`	11	Success, skip, retry and failure decisions; recurring tasks stay registered; backoff limits
`TimeUtilsTest`	6	Natural-language time parsing and next-run delay
`MailGatewayTest`	2	Demo drafts and sent mail survive a restart and reset restores the sample; the gateway routes by source and raises a revision signal on every change
`DemoMailServiceTest`	6	Gmail-style operators, ordering and caps, read and 404, drafts and sent folders, idempotent send
`ExecutionLogFormatterTest`	6	Raw errors become plain language with a next step
`MemoryToolTest`	4	Upsert by key, update, delete, ambiguous delete deletes nothing, recall
`EmailTaskContextTest`	4	Email-task detection and safe instructions
`BrowserUrlTest`	5	Domain, http upgrade, search fallback, blank input, host display
`GmailInboxToolsTest`	3	Result formatting, no invented data, transient versus permanent error classification
`ControlledWebScraperTest`	3	Public-page extraction and private-address blocking
`MarkdownParserTest`	3	Bold, italic, code, lists, malformed markers
`SchedulerToolTest`	5	The model's instruction is stored unchanged; invalid time rejected; weekday, specific-day, cron and one-time recurrences; bad patterns rejected
`CronScheduleTest`	10	Cron parsing (lists, ranges, steps, names, Sunday as 0 or 7), next-run calculation (today vs tomorrow, weekends skipped, month and day-of-month rules, classic day-of-month OR day-of-week), invalid input rejected, legacy daily format, friendly recurrence words, changing the clock while keeping the days
`MetricMathTest`	2	Percentiles and summary statistics
Total	**99**	

2.2 Backend (`npm test`)

Suite	Tests	What it proves
`shared-account.test.js`	4	Test account detection; it may send only to itself or reserved example addresses (real third parties rejected); it cannot be disconnected; normal accounts are unrestricted

Suite	Tests	What it proves
`gmail_inbox.test.js`	7	MIME body extraction, HTML-to-text, header lookup, raw draft building and header-injection rejection, Google error mapping, result cap, disconnected-account error
`validation.test.js`	3	Recipient, time zone and offset validation; injection rejected
`token_vault.test.js`	2	Refresh-token encryption and tamper detection
`worker.test.js`	2	Durable queue provisioning
Total	**18**	

2.3 Database migration

The `MIGRATION_1_2` SQL was compared against the schema Room generates for the new tables. They match exactly. On a physical device an upgrade install kept the existing chat history and memories.

3. Device test log

Tested on the physical device against the deployed backend.

#	Requirement	Steps	Expected	Result
D1	FR-18	Settings, Connect Gmail, choose account, grant	Connected as the account	Pass
D2	FR-14	Ask for email with an old session that lacks the inbox scope	Clear "reconnect Gmail" message, no invented data	Pass
D3	FR-03, 04, 12, 15	"Check my emails from the last 7 days and summarize them"	`search_emails` runs; summary of real messages	Pass
D4	FR-03, 16	"Draft an email to my professor asking for an extension. Do not send it."	Asks for the missing address, then saves a formal draft, nothing sent	Pass
D5	FR-08	"Send an email to the test account…"	Approval dialog; after approval the mail is delivered	Pass (found by a follow-up inbox search)
D6	FR-06	Saved preference "formal emails", then drafted	Draft is formal	Pass
D7	FR-05	Save a memory, force-stop, relaunch	Memory still listed	Pass
D8	FR-07	Delete a memory by voice command and from the screen	Removed; screen asks for confirmation	Pass
D9	FR-10, 11, 13	"Every morning at 9 AM…", then Run now	Daily task created; Success row in History; notification	Pass
D10	FR-09	"Open news.ycombinator.com and tell me the top 3 stories"	Browser opens, top stories listed	Pass
D11	FR-17	Demo mailbox on, "find interview invitations and summarize"	Search and read run on sample data; accurate summary	Pass
D12	FR-21	Address bar with a domain; browser controls	Page loads; controls respond	Pass
D13	FR-22	Open a Wikipedia article, tap Summarize	Returns to chat with a structured summary of that page	Pass
D14	FR-23	Theme selector System, Light, Dark	Applies immediately	Pass
D15	FR-24	First launch	Splash animation then onboarding	Pass
D16	FR-25	Change the default recipient email and save	New value shown and kept	Pass
D24	FR-08	Test account: "Send an email to the test address…", tap Send email	Approval dialog; card "Email queued"; the message arrives in Gmail	Pass (Gmail screenshots in Email Flow)
D25	FR-16, FR-06	Test account: "Draft a formal email to Meera Rao… do not send it"	Draft saved in Gmail Drafts, formal wording, nothing sent	Pass (Gmail screenshots)
D26	FR-21	Browser: type words in the address bar; open the menu; bookmark; open Bookmarks and History; Reader view	Search results; menu with Find, Reader, Desktop, text size, Share, Copy, Open in Chrome; the bookmark is listed	Pass
D27	FR-09, FR-22	"Open en.wikipedia.org/wiki/Mumbai and read the page", then tap Key facts	The agent opens the browser by itself; the chat returns a structured, dated fact list	Pass
D28	NFR-10	Simulated small phone (360 x 640 dp): onboarding, chat, Settings, Tasks, browser	No clipped or unreachable content; the onboarding mailbox page scrolls so every button is reachable	Pass
D29	NFR-10	Accessibility font scale 1.6 on the small screen	After fixes: title stays on one line, bottom tab labels do not break, the message hint truncates cleanly	Pass
D30	NFR-10	Simulated tablet width (600 x 960 dp)	Content stays centered at a readable width (max 640 dp) instead of stretching edge to edge	Pass

#	Requirement	Steps	Expected	Result
D31	FR-10, FR-11	"Every weekday at 5:14 PM, check my important emails and summarize them"	Task card "Weekdays at 17:14"; it runs at 17:14 by itself through WorkManager (11.5 s); History shows Success; the next run is **Monday 16 Nov 17:14**, skipping the weekend	Pass
D32	FR-10	Existing daily task created by an earlier build	Still shows "Daily at 09:00" with the next run time after the upgrade (migrated to the cron chain)	Pass
D17	NFR-01	30 simple prompts	Median 1.6 s, on-device overhead about 0.1 s	Pass
D18	FR-27, 28	Open the mailbox viewer in demo mode	Inbox with unread in bold; tabs for Sent and Drafts	Pass
D19	FR-27	Ask Aurviz to draft an email, then to send one, then open the Drafts and Sent tabs	The Drafts and Sent tabs show the new messages	Pass
D20	FR-28	Settings, Mailbox: select Demo mailbox	Header badge DEMO; the agent uses the sample inbox	Pass
D21	FR-26	Settings, Mailbox, Test Gmail account	Email, masked password with show/copy, six numbered steps	Pass
D23	FR-26	Follow the steps: Sign in with Google, choose the test account, Continue past the unverified-app warning	Back in Aurviz showing "Connected as adityatesting404@gmail.com"; "Search my inbox" returns 15 real messages and a correct summary	Pass
D22	FR-20	History after a manual run	Plain headline, duration and formatted summary	Pass

Defects found during device testing, and fixed

Defect	Fix
Large font sizes broke the header ("Aurvi/z"), the bottom tabs ("Memor/y") and the message hint	One-line titles and labels, compact tab text, ellipsized hint; verified at font scale 1.6
Onboarding pages could cut off buttons on short screens	Pages now scroll when they do not fit (`CenteredScroll`)
Layout stretched across very wide screens	Content column capped at 640 dp and centered
A background task could pop the browser screen open or wait for it	Screen-bound tools are refused in background runs; regression test added
Agent tab showed Settings after opening Settings from the sign-in banner (tab back stacks nested)	Links use the same tab navigation as the bottom bar
Search label showed `'{}'` when the model sent an empty query	Symbol-only queries mean "recent inbox"
Scheduler rewrote the model's instruction by keyword matching ("summarize" became "draft responses")	Instruction now stored unchanged; regression test added
Background email tasks told the model the wrong account address	Removed the claim; the tools use the connected account
A one-off network error left the Gmail status stuck on screen	Status re-checks on opening Settings; Retry button added
Model sometimes returned a lone "-" as its final answer	Token limit raised; one automatic retry; then a retryable error
Cold-start of the free-tier backend caused "could not reach"	Warm-up request at launch; timeouts allow a 45 s wake-up
Status-bar text invisible on light backgrounds	Icons now follow the active theme

4. Requirement-to-evidence matrix

Requirement	Automated	Device
FR-01 Voice input	n/a (platform API)	Earlier build
FR-02 Text input	`AgentOrchestratorTest`	D3
FR-03 Intent detection	`AgentReliabilityTest`	D3, D4
FR-04 Tool calling	`AgentOrchestratorTest` , `GeminiAgentClientTest`	D3
FR-05 Persistent memory	`MemoryToolTest`	D7
FR-06 Memory retrieval	`AgentReliabilityTest` , `MemoryToolTest`	D6
FR-07 Memory edit/delete	`MemoryToolTest`	D8
FR-08 Send after confirmation	`AgentOrchestratorTest`	D5
FR-09 Web automation	`ControlledWebScraperTest` , `BrowserUrlTest`	D10
FR-10 Scheduling	`CronScheduleTest` , `SchedulerToolTest` , `TimeUtilsTest`	D9, D31, D32
FR-11 Background execution	`TaskExecutionPolicyTest`	D9 (Run now through WorkManager)
FR-12 Multi-step	`AgentReliabilityTest`	D3, D11
FR-13 Task history	n/a	D9
FR-14 Error reporting	`AgentReliabilityTest` , `ExecutionLogFormatterTest`	D2
FR-15 Search and read	`DemoMailServiceTest` , `gmail-inbox.test.js`	D3, D11
FR-16 Drafts	`GmailInboxToolsTest`	D4

Requirement	Automated	Device
FR-17 Demo mailbox	`DemoMailServiceTest`	D11
FR-18 OAuth	`token-vault.test.js`	D1
FR-19 Task management	`TaskExecutionPolicyTest`	D9
FR-20 Readable results	`ExecutionLogFormatterTest`	Screen check
FR-21 Browser tools	`BrowserUrlTest`	D12, D26
FR-22 AI on pages	n/a	D13, D27
FR-23 Theme	n/a	D14
FR-24 Onboarding, shortcuts	n/a	D15
FR-25 Settings	n/a	D16
FR-26 Test Gmail account	`shared-account.test.js`	D21, D23
FR-27 Mailbox viewer	`MailGatewayTest`	D18, D19
FR-28 Mailbox selection	`MailGatewayTest`	D20
NFR-01 Performance	`MetricMathTest`	D17
NFR-02 Security	`token-vault.test.js`, `validation.test.js`	D1, D5
NFR-03 Reliability	`TaskExecutionPolicyTest`, `AgentReliabilityTest`	D9
NFR-07 Fault tolerance	`AgentReliabilityTest`	D2

5. How to reproduce

```
./gradlew app:testDebugUnitTest        # 99 Android tests
./gradlew app:assembleDebug            # build the APK
cd backend && npm ci && npm test       # 18 backend tests
```

Device checks follow the Evaluator Guide.

Performance

Method

Aurviz measures itself. `PerformanceRecorder` writes one row per sample into Room (newest 1000 kept), and **Settings, Performance** shows count, median, 95th percentile and success rate per metric.

Metric	What is timed
Agent turn (end-to-end)	Prompt submitted until the answer or error is shown. Includes Gemini and Gmail network time, and any time spent waiting on an approval dialog
Gemini call	One `generateContent` round trip, including model fallback
Memory retrieval	The Room lookup that builds the memory context
Tool: `<name>`	One tool execution, for example `search_emails`
Scheduled task run	One background WorkManager run, start to persisted result

Percentiles use the nearest-rank method (`MetricMath`, unit-tested).

Results (13 November 2026)

Device: realme RMX5116, Android 16 (API 36), Wi-Fi, debug build, model `gemini-3.5-flash-lite`. **Workload:** 30 simple one-line prompts needing no tools, sent through the UI after resetting the samples.

Metric	n	Median	p95	Success
Agent turn (end-to-end)	30	1.6 s	6.6 s	100%
Gemini call	30	1.5 s	6.4 s	100%
Memory retrieval (Room)	30	54 ms	93 ms	100%

Reading the numbers against the target

The target is under 3 s for simple commands, excluding external service time.

- **On-device overhead is about 0.1 s at the median** (agent turn minus Gemini call). That covers memory retrieval, prompt assembly, tool dispatch, persistence and UI state.
- **End-to-end median is 1.6 s**, below 3 s even including the network round trip.
- **The 6.6 s p95 is almost entirely Gemini's own latency** (its p95 is 6.4 s): external service time, not on-device work.

Other single observations: `search_emails` about 0.9 s for one call, `manage_memory` 43 ms, and a manually triggered background run that searched Gmail and summarized took 9.9 s end-to-end (several model and tool steps, each adding a Gemini round trip).

Limitations of this study

One device, one network, one time of day, simple prompts. Multi-step email workflows take proportionally longer. No battery or long-run memory study was performed. WorkManager timing is not exact, so scheduled-run delay was not benchmarked.

Task execution latency (scheduled runs)

Measured by the app's recorder on the same device. A scheduled run is a full background agent run: several model calls and tool calls through WorkManager.

Run	Duration	Notes
Email summary, real Gmail via the backend (run 1)	9.9 s	Search, read, summarize
Email summary, real Gmail via the backend (run 2)	21.3 s	More messages read, more model steps
Email summary, demo mailbox	6.9 s	Local mail tools, no network hop to Gmail
Weekday task fired by the scheduler at 17:14 (demo mailbox)	11.5 s	Started on the minute; shows scheduling accuracy on a charging, awake phone

Task latency is dominated by the number of model round trips (about 1.5 s each) plus Gmail calls; it is a background job, so the user never waits on it. WorkManager's scheduling delay (how late Android starts a run) depends on battery state and was not benchmarked.

Memory usage

`dumpsys meminfo` on the physical device, debug build, after a session that included chat, the mailbox viewer and the browser:

Measure	Value
Total PSS	165 MB
Java heap	18 MB
Native heap	16 MB
Graphics	39 MB
Resident set (RSS)	289 MB
Swap PSS	0.1 MB

Most of the footprint is graphics and the WebView that backs the browser (created only while the browser screen is open and destroyed afterwards). The Kotlin heap is small because chat history is capped at 100 messages and results are bounded. This is a single point-in-time measurement on a debug build, not a peak or a leak test.

Optimization implemented

Bounded model context. Each turn sends at most 12 prior messages, 8 memories, email bodies capped at 6,000 characters and 15 search results. This keeps requests small, which lowers latency and token cost, and it keeps the app's memory flat over long sessions. It is a design choice; its effect on latency was not isolated in an A/B measurement.

Backend warm-up. The free-tier server sleeps when idle and needs about 30 to 50 seconds to wake. The app sends a health request at launch and allows a 45-second first request, so the first Gmail action no longer fails with "could not reach".

Optimization opportunities (not yet done)

1. **Stream the model response.** Gemini supports streaming. Showing text as it arrives would make the 1.5 s median feel instant, and it would cut perceived latency for long summaries where the 95th percentile reaches 6.6 s.
2. **Cache and parallelize Gmail reads.** `search_emails` already fetches message metadata in parallel on the server. `read_email` calls are sequential through the agent loop; letting the model request several reads at once (parallel function calls) would cut a multi-email summary from about N model round trips to about two.
3. **Memory retrieval index.** Replace the per-term `LIKE` queries with an FTS4 table (or embeddings) once memories grow into the thousands.
4. **Shrink the APK and cold start.** Enable R8 minification and a baseline profile for the release build (minification is off in the current debug and release configurations).

Reproduce

Settings, Performance, Reset samples, send at least 30 prompts, read the table.

Resource-efficiency choices

- No foreground or persistent background service. Deferred work uses WorkManager with a network constraint, so it never wakes the device just to fail offline.
- Bounded context: 12 prior messages, at most 8 memories, email bodies capped at 6,000 characters, search capped at 15 results, at most 8 interactive and 6 background model steps.
- Bounded tables: performance 1000 rows, task history 500 rows, chat 100 messages, browser history 60 pages.
- The hosted backend is warmed at launch to hide cold starts.

Evaluator Guide

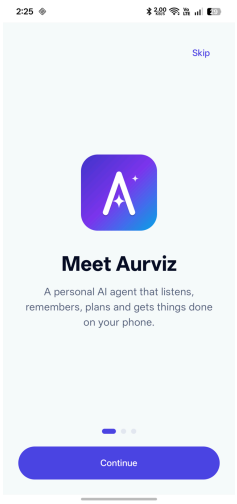
Test every feature of Aurviz in about ten minutes. **You do not need a password, a Google account or any setup.** Aurviz offers three mailboxes, and the credentials of a dedicated test Gmail account are shown inside the app.

1. Install and first launch

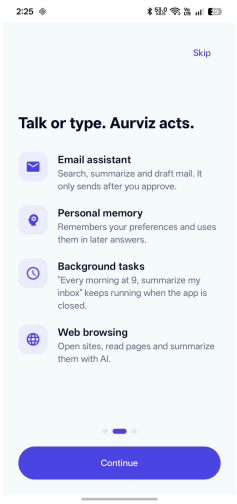
1. Install the APK on an Android 8.0 or newer phone (allow "install unknown apps" for your file manager or browser).
2. Open **Aurviz**. You see the animated logo, then a short walkthrough.
3. On the last page tap **Show me the sign-in steps**. Settings opens with the test account's **email and password** (tap the copy icons) and six numbered steps. Follow them (about one minute).
4. Allow the microphone and notification permissions when asked.



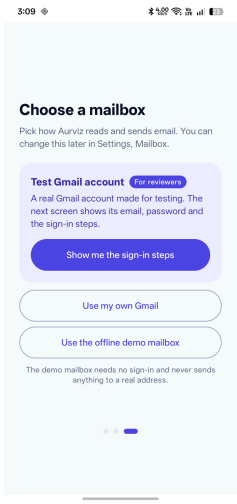
Animated splash



Welcome



What it can do



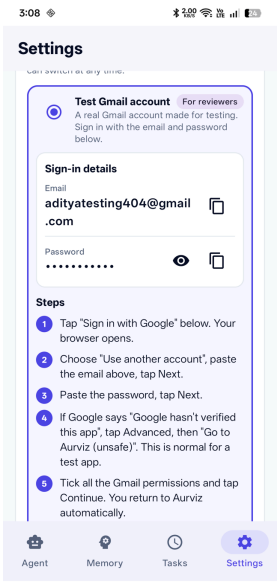
Choose a mailbox

The AI key is built into the APK. If your build has none, open **Settings, AI model** and paste a free Gemini key from https://aistudio.google.com/.

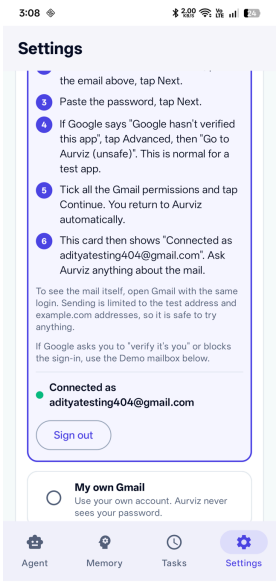
2. The three mailboxes

Mailbox	What it is	Credentials needed	Best for
Test Gmail account (default)	Real Gmail on a dedicated throwaway account (`adityatesting404@gmail.com`)	Email and password are shown in Settings, Mailbox, with copy buttons	Seeing the real Gmail integration
My own Gmail	Your own account through Google's sign-in page	Your own Google account (must be an OAuth test user, see section 6)	Using your own mail
Demo mailbox	Ten sample emails stored on the phone. Fully offline	None	Trying everything with zero risk, including sending

Switch at any time in **Settings, Mailbox**. The header badge shows which one is active (**TEST ACCOUNT** or **DEMO**).



Test account: email and password



Six numbered sign-in steps

Signing in to the test account (step by step)

1. **Settings, Mailbox, Test Gmail account.** The email and password are shown; use the copy icons.
2. Tap **Sign in with Google**. Your browser opens.
3. Choose **Use another account**, paste the email, **Next**, paste the password, **Next**.
4. Google shows **"Google hasn't verified this app"**. This is normal for a test app: tap **Continue** (or Advanced, then Go to Aurviz).
5. Tick the Gmail permissions and tap **Continue**. You return to Aurviz, and the card shows **Connected as adityatesting404@gmail.com**.
6. Ask Aurviz anything. To see the mail itself, open Gmail with the same login.

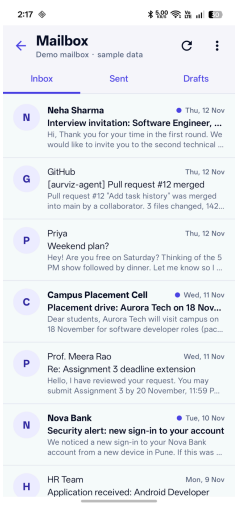
If Google asks to "verify it's you" or blocks the sign-in (it can do this for a shared account), use the **Demo mailbox**.

Why the test account is safe to share

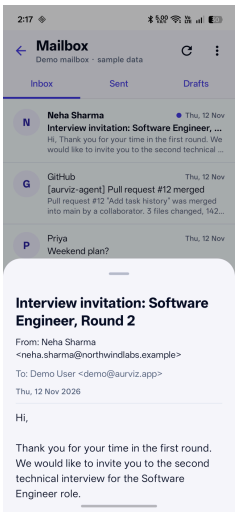
It is a throwaway account with no private data. On the server it **can only send mail to itself or to example.com / example.org / example.net addresses** (which reach no real person), and it **cannot be disconnected** from the app, so one reviewer cannot break it for another.

3. See the demo mailbox

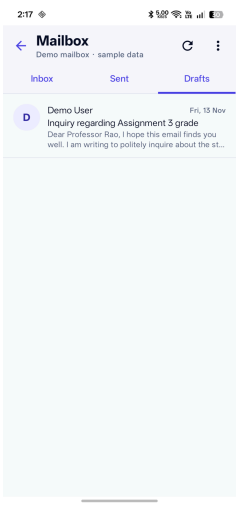
The in-app mailbox viewer exists **only for the demo mailbox**. Real accounts are viewed in Gmail itself. With the demo mailbox selected, tap the **inbox icon** in the chat header (or **Settings, View demo mailbox**). The viewer shows **Inbox, Sent and Drafts**, with unread messages in bold. Tap a message to read it, then **Summarize with Aurviz**. In demo mode the viewer **updates live** as Aurviz drafts or sends, and the menu has **Reset demo mailbox**.



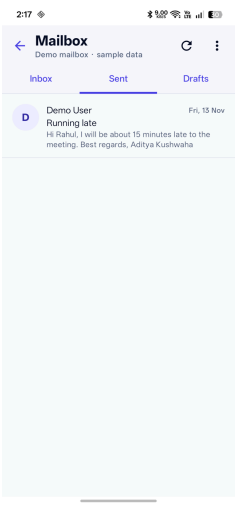
Inbox



Open a message



Drafts update live

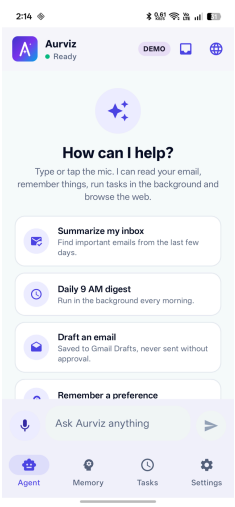


Sent updates live

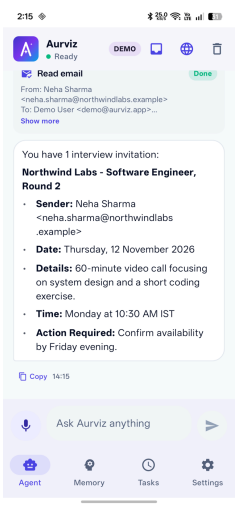
4. Ten-minute test script

Type each prompt, or tap the microphone and say it.

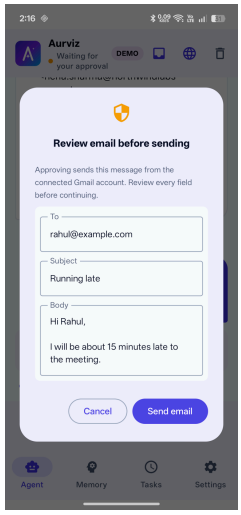
#	Say or type	What you should see	Requirement
1	"Check my emails, find interview invitations and summarize them"	Cards "Search Gmail" then "Read email", then a structured summary (in demo: the Northwind Labs interview)	FR-03, 04, 12, 15
2	"Which of my emails are important and unread?"	The important, unread messages	FR-15
3	"Remember that I prefer formal emails"	A memory card; the Memory tab lists it	FR-05
4	"Draft an email to meera.rao@university.example asking about my grade, but don't send it"	A formal draft is saved; nothing is sent. Open the **Drafts** tab to see it	FR-06, 16
5	"Send an email to rahul@example.com saying I will be late"	An **approval dialog** with editable To, Subject and Body. Cancel sends nothing. Approve and it appears in **Sent** (the test account accepts only its own address and example.com addresses; the demo mailbox accepts any)	FR-08
6	"Every weekday at 9 AM, check my important emails and summarize them" (also try "every Monday and Thursday at 6 PM")	Task created and shown in words, for example **Weekdays at 09:00**, with its next run time. Open **Tasks**, tap **Run now**; **History** shows the run	FR-10, 11, 13
7	"Forget my email tone preference"	Memory deleted	FR-07
8	"Open news.ycombinator.com and tell me the top 3 stories"	The in-app browser opens and the stories are listed	FR-09
9	In the browser open a Wikipedia article and tap **Summarize**	Back in chat with a summary of that page	FR-22



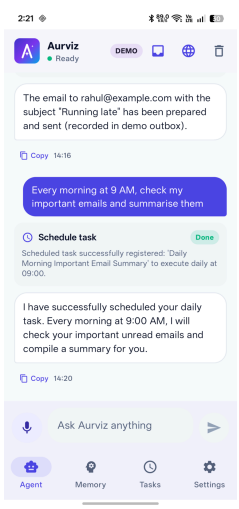
Start screen



Search, read, summarize



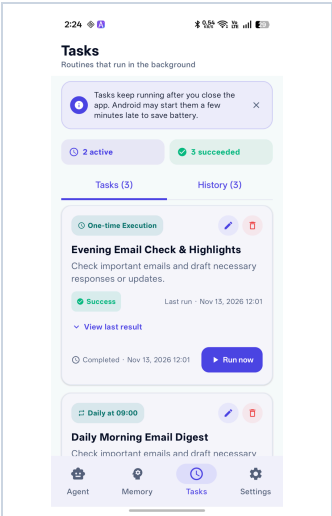
Approval before sending



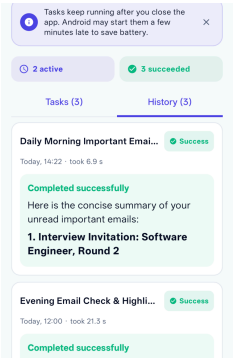
Scheduling by voice or text

5. More to explore

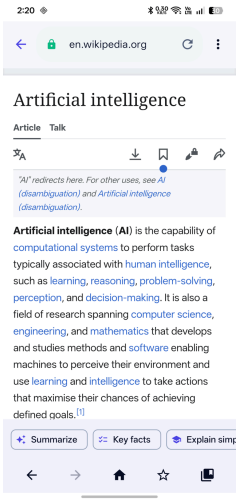
- **Tasks:** pause and resume a daily task. **History** shows each run with its duration, and failures are explained in plain words with a next step.
- **Browser (globe icon):** address bar that also searches, find in page, reader view, desktop site, text size, bookmarks, history, share. The AI buttons Summarize, Key facts, Explain simply and Ask about this page use the page you are viewing.
- **Settings, Appearance:** System, Light or Dark.
- **Settings, Performance:** latency measured on your device (median and p95).
- **Long-press the app icon:** shortcuts for Voice command, Summarize inbox and Tasks.
- **Share text** from any app to Aurviz to get a summary.
- **Failure handling:** switch on airplane mode and send a prompt. You get a clear message and a **Try again** button.



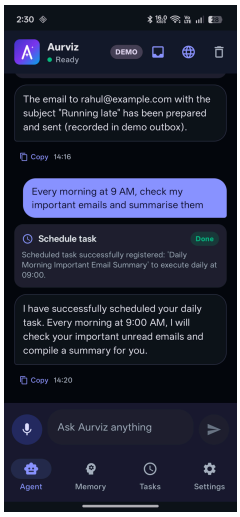
Tasks



Readable history



Browser with AI actions



Dark theme

6. Using your own Gmail (optional)

Google allows only listed accounts to grant the Gmail read permission while an app is in testing mode. The project owner can either:

1. **Add your account as a test user**: Google Cloud Console, APIs and Services, OAuth consent screen, Audience, Test users. Then choose **My own Gmail** in Settings and sign in on Google's own page. You never share a password with Aurviz or anyone.
2. **Publish the consent screen "In production"** (unverified). Any Google user can then continue through the "Google hasn't verified this app" warning, limited to 100 users until Google verifies the app.

7. Deeper reading

- Email Flow: the real-Gmail path step by step, with the Aurviz screens next to the matching Gmail screens as proof.
- Browser: every browser feature, with screenshots.

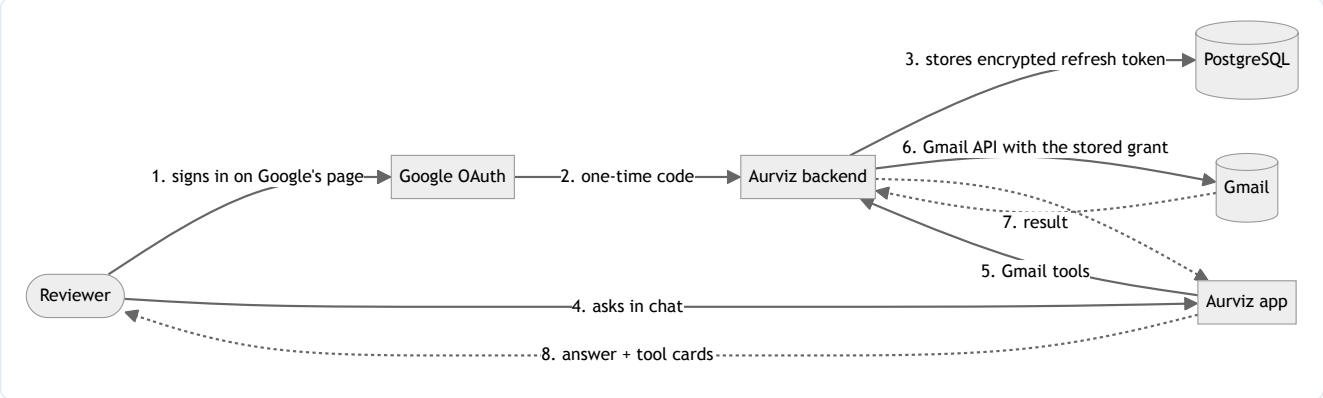
8. Mapping to common evaluation criteria

Criterion	Where to look
AI reasoning and tool calling	Scripts 1, 4, 5; the tool cards in chat; Architecture, the agent loop
Modular architecture	Architecture; the `domain`, `data`, `presentation`, `service` packages
Persistent memory	Scripts 3, 4, 7; restart the app and open the Memory tab
Background scheduling	Script 6; Architecture, scheduled tasks
OAuth and secure email	Section 2 and 6; Security
Error handling and reliability	Airplane-mode test; Tasks, History; Test Report
Performance measurement	Settings, Performance; Performance
UI and UX	The whole app: dark mode, onboarding, animations, mailbox viewer

Email Flow: How Aurviz Reads and Sends Real Gmail

This document shows, end to end and with evidence, how Aurviz works with a **real Gmail account**. The reviewer test account is `adityatesting404@gmail.com`, a throwaway account created only for evaluation. Every Aurviz screenshot below was taken on a physical phone, and every Gmail screenshot was taken in the Gmail app of the same account, so the result of each action can be checked independently.

1. The idea in one picture

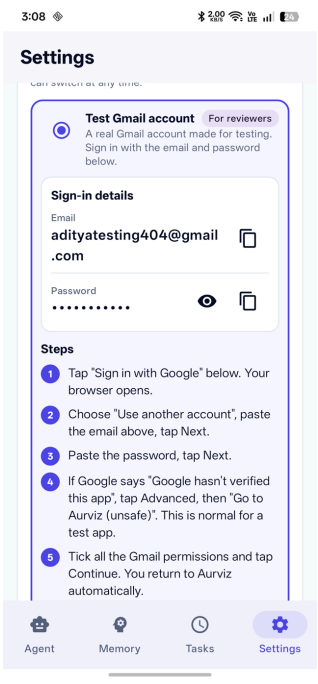


Key points:

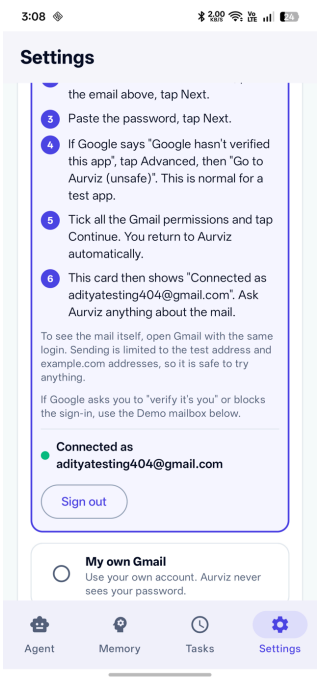
- **No password ever touches Aurviz.** The reviewer types the email and password only on Google's own sign-in page.
- **The phone never holds a Google token.** The backend keeps an AES-256-GCM encrypted refresh token; the phone only holds an opaque session token.
- **Sending is always approved by a person**, and background tasks can never send.

2. Signing in (about one minute)

Open **Settings, Mailbox, Test Gmail account**. The email and password are shown with copy buttons, followed by six numbered steps.



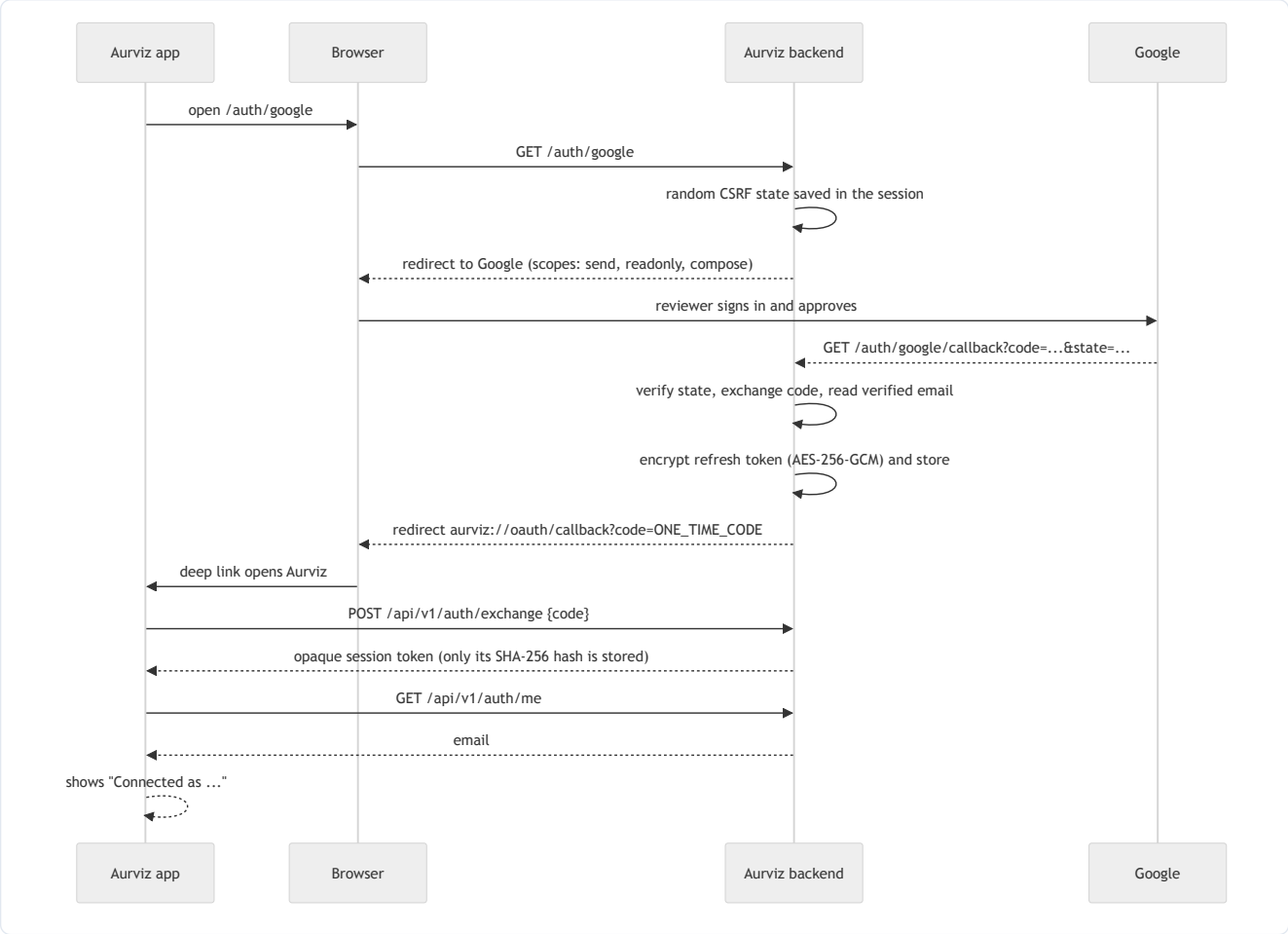
Email, hidden password with show and copy buttons



Six numbered steps and the status line

1. Tap **Sign in with Google**. The browser opens Google's sign-in.
2. Choose **Use another account**, paste the email, **Next**.
3. Paste the password, **Next**.
4. Google shows **"Google hasn't verified this app"**. This is expected for a test application. Tap **Continue** (or Advanced, then Go to Aurviz).
5. Tick the Gmail permissions and tap **Continue**. You return to Aurviz automatically.
6. The card now reads **Connected as adityatesting404@gmail.com**.

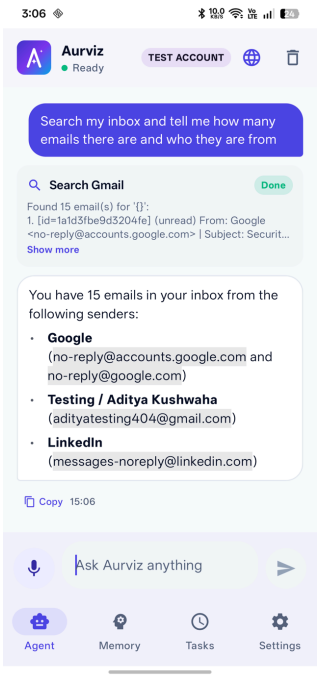
What happens behind the scenes



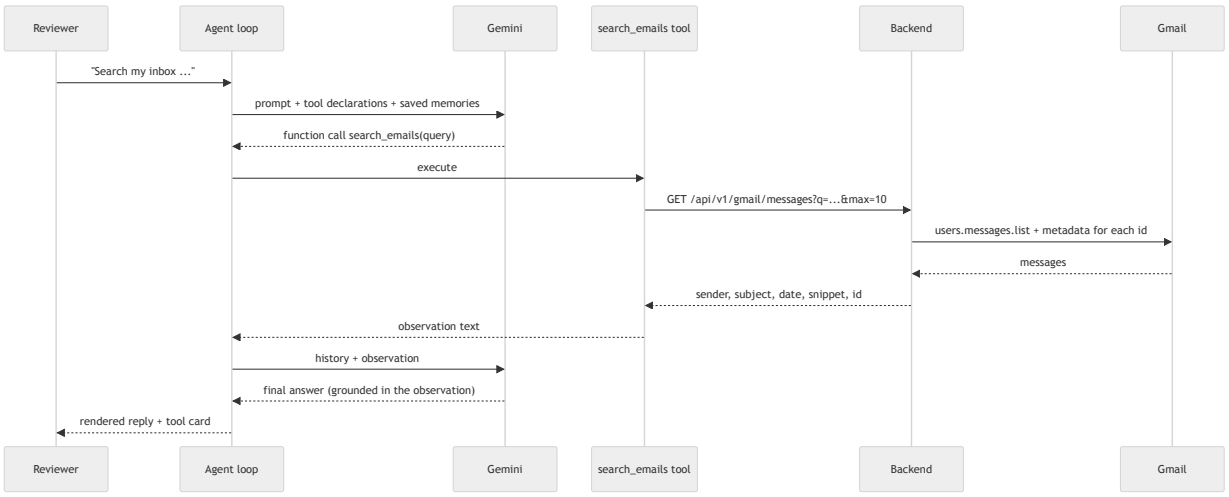
Security properties that are visible in this flow: CSRF `state` check, single-use one-time code (5 minutes), hashed session tokens, encrypted refresh tokens, and no Google credentials in the APK.

3. Reading the real inbox

Ask: "*Search my inbox and tell me how many emails there are and who they are from.*" Aurviz calls the `search_emails` tool, which goes to the backend and then to Gmail. The tool card shows what ran; the answer is built only from the returned messages.



Aurviz: search_emails on the real inbox, then a summary

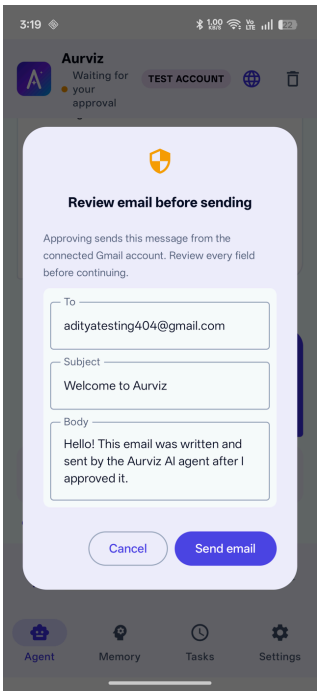


The model is told never to invent mail content. If a search returns nothing, the tool says so plainly.

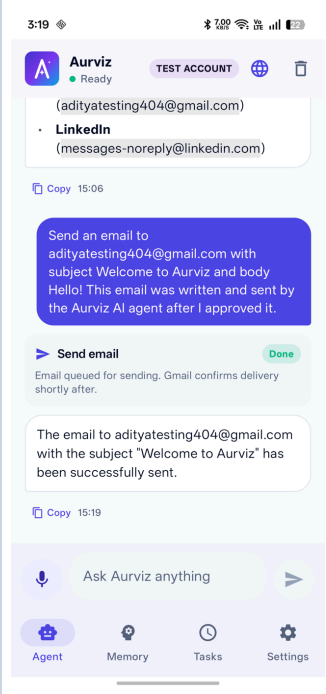
4. Sending an email (with approval)

Ask: "*Send an email to adityatesting404@gmail.com with subject Welcome to Aurviz and body Hello! This email was written and sent by the Aurviz AI agent after I approved it.*"

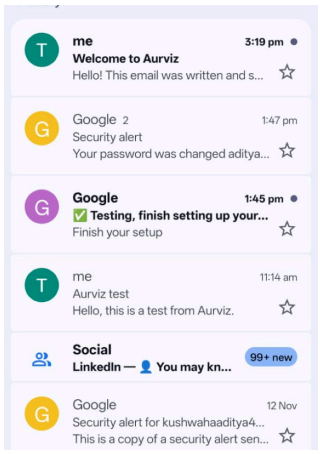
Aurviz does **not** send yet. It pauses the agent loop and shows an editable **approval dialog** (To, Subject, Body). Only the **Send email** button releases the send. Cancel sends nothing.



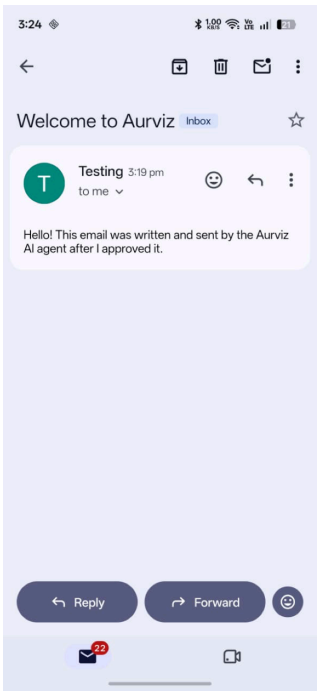
1. Approval dialog, fields are editable



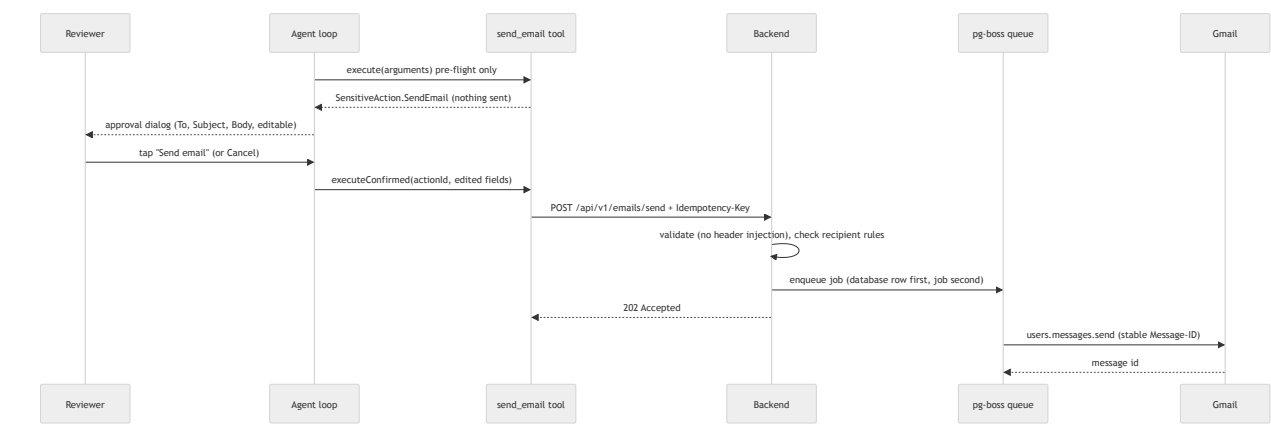
2. After approval: tool card "Email queued for sending"



3. Gmail: "Welcome to Aurviz" has arrived



4. Gmail: the email opened, with the body the agent wrote

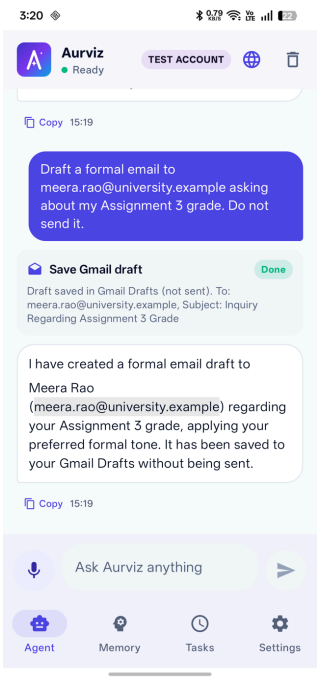


What makes the send safe and reliable:

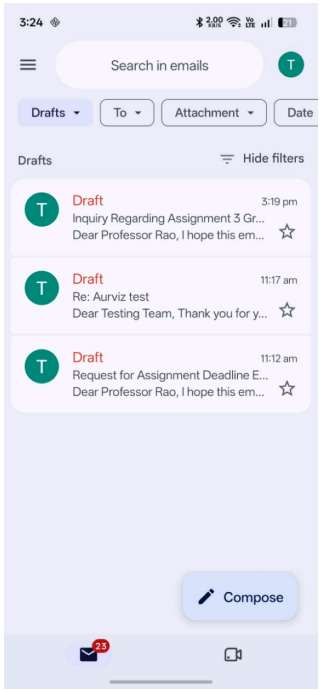
- **Approval is enforced in code**, not by the prompt. A tool marked sensitive cannot run without `executeConfirmed`.
- **Idempotency key**: tapping twice or retrying never sends two copies (a unique database constraint).
- **Header-injection protection**: line breaks in recipient or subject are rejected in the app and again on the server.
- **Honest delivery wording**: the card says "queued"; Gmail confirms delivery moments later, as the inbox screenshot shows. Ambiguous network outcomes are stored as `UNKNOWN` and never resent automatically.
- **Test-account guardrails (server side)**: the shared account can send only to itself or to `example.com`, `.org`, `.net`, `*.example` and `*.test` addresses (which reach no real person), and it cannot be disconnected.

5. Drafts: "prepare it, but don't send it"

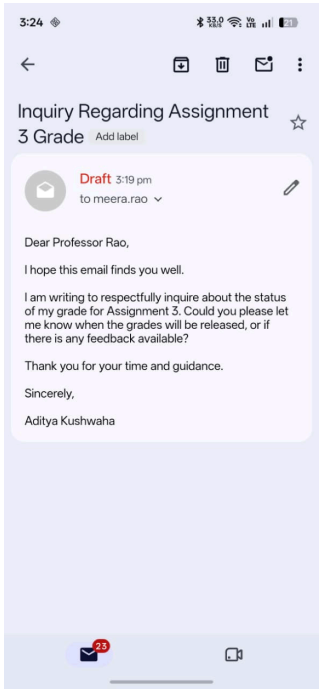
Ask: "*Draft a formal email to meera.rao@university.example asking about my Assignment 3 grade. Do not send it.*" The agent chooses `create_email_draft` instead of `send_email`, applies the saved preference for formal tone, and saves the draft in Gmail. There is no approval dialog because nothing leaves the account.



Aurviz: "Draft saved in Gmail Drafts (not sent)"



Gmail: the Drafts folder shows the new draft at the top



Gmail: the draft opened, formal wording, signed with the user's name

The two older drafts visible in the Drafts folder came from earlier test runs of the same feature (a deadline-extension request and a reply to the first test email). They show the same behaviour on different prompts.

6. How the pieces map to requirements

Step	Requirement
OAuth sign-in, no password in the app	FR-18, NFR-02
Search and read mail with Gmail operators	FR-15, FR-04
Multi-step: search, read, summarize	FR-12
Approval before sending	FR-08
Drafts that are never sent	FR-16
Tool cards and plain errors	FR-14, FR-20
Restricted shared account	FR-26, NFR-12

7. No account? Use the demo mailbox

If Google asks to verify the sign-in (a shared account used from many phones can trigger this), choose **Settings, Mailbox, Demo mailbox**. It offers the same search, read, draft and send features on ten sample emails stored on the phone, and the in-app viewer shows Inbox, Sent and Drafts updating live. See the Evaluator Guide.

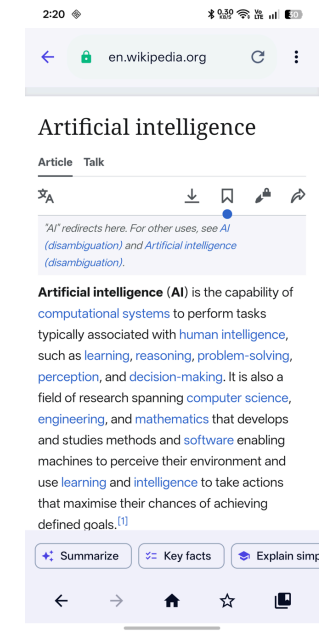
The Aurviz Browser

Aurviz includes a controlled in-app browser. It serves two users at once: **people** get a real, comfortable browser, and the **AI agent** drives the same page through a small set of tools. Because both share one page, you can ask Aurviz to open a site, then keep reading and use the page's content with AI.

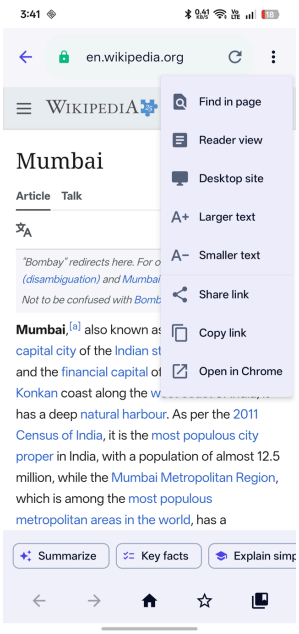
Open it with the globe icon in the chat header.

1. Features at a glance

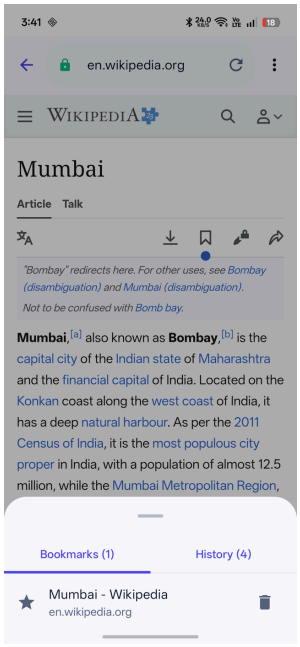
Area	What you get
Address bar	Type a website **or just words**: words become a web search
Page tools	Find in page, **Reader view**, Desktop site, Larger and Smaller text
Navigation	Back, forward, home, reload and stop, progress bar
Library	**Bookmarks** (star button) and **History**, stored on the phone
Sharing	Share link, Copy link, Open in Chrome
AI on the page	**Summarize**, **Key facts**, **Explain simply**, **Ask about this page**
Agent tools	`open_url` , `search_webpage` , `read_page` , `extract_links` , `scroll_page` , `navigate_back/forward` , `get_browser_state` , and approved `click_element` and `enter_text`
Safety	HTTPS only, private and local addresses blocked, no JavaScript bridge, approval for anything that changes a page



Address bar, secure lock, AI action chips, bottom toolbar



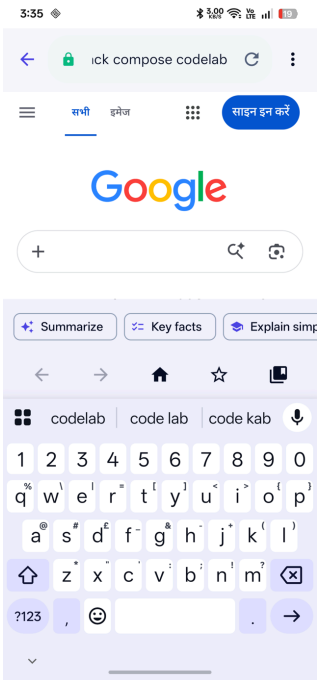
Page tools menu



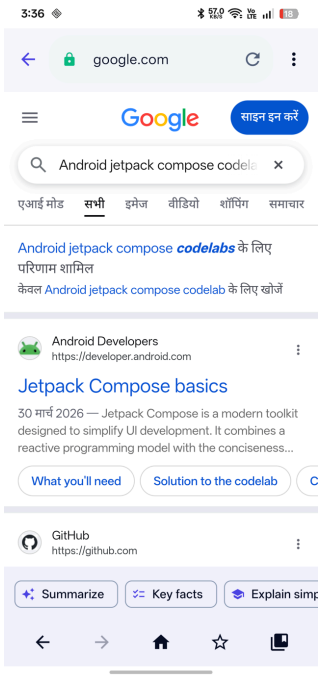
Bookmarks and History sheet

2. Search from the address bar

The address bar understands what you type. `news.ycombinator.com` becomes `https://news.ycombinator.com` ; `http://` is upgraded to `https://` ; plain words such as `android jetpack compose codelab` become a web search. This logic is a pure function (`BrowserUrl.resolve`) covered by unit tests.



Typing words, not a URL



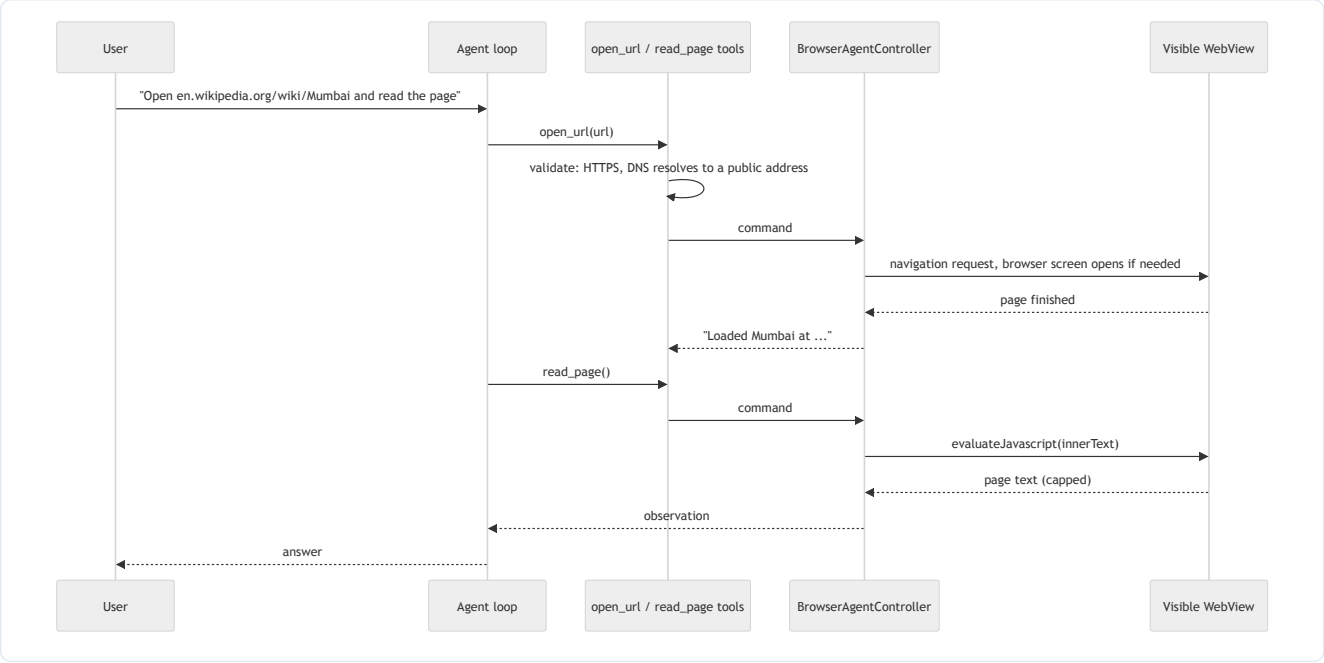
The search runs and results load

3. Agent-driven browsing

Ask in chat: "*Open en.wikipedia.org/wiki/Mumbai and read the page.*" The agent calls `open_url`, the browser screen opens by itself, the page loads, and `read_page` returns the text to the model. The page stays on screen for you to keep using.



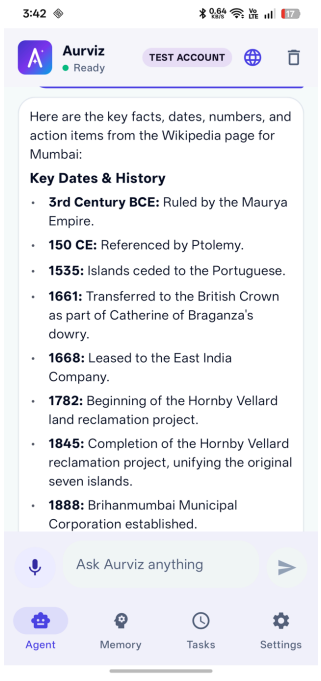
Opened by the agent, ready for you to scroll, bookmark or ask about



4. AI on whatever page you are viewing

Four chips sit above the toolbar. Each one reads the current page text, sends it with a fixed instruction, and returns you to the chat with the answer:

- **Summarize**: a short bullet summary.
- **Key facts**: dates, numbers and action items.
- **Explain simply**: the page in beginner-friendly words.
- **Ask about this page**: type your own question, for example "What is the deadline mentioned here?"

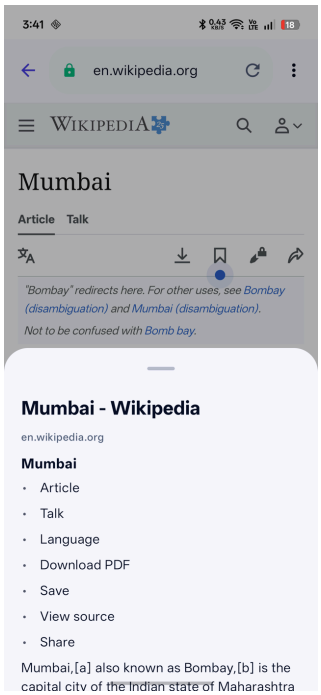


"Key facts" on the Mumbai article: dated, structured, formatted

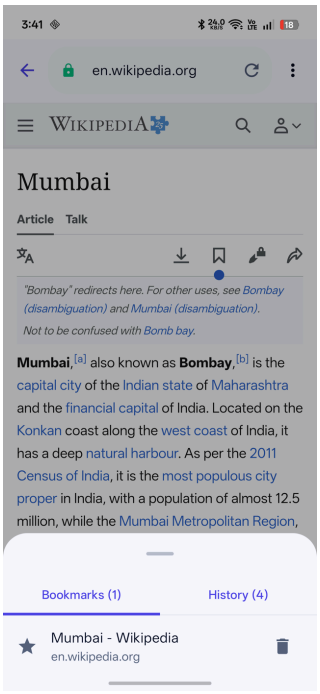
Privacy: page text is sent to the model **only when you tap one of these actions**, it is capped in size, and it is marked as untrusted data so instructions hidden inside a web page are not obeyed (prompt-injection defence).

5. Reader view, bookmarks and history

Reader view extracts the article text (headings, paragraphs and long list items) into a clean sheet, skipping menus, info boxes and references, with a **Summarize with Aurviz** button.



Reader view sheet



A bookmarked page; the History tab lists visited pages

Bookmarks and history are stored locally (history keeps the last 60 pages and can be cleared). Nothing is uploaded.

6. Safety model

Risk	Control
Opening dangerous or internal addresses	HTTPS only; `localhost`, private, link-local and unique-local ranges rejected; the agent's `open_url` also resolves DNS before loading
Page scripts talking to the app	No `addJavascriptInterface`; file and content access disabled; mixed content blocked; Safe Browsing on
The agent changing a page	`click_element` and `enter_text` pause for an approval dialog showing the page, the selector and the text; typing never submits a form
Hidden instructions in page text	Page content passed to the model is labelled untrusted and is only sent on user action
Background misuse	Screen-bound tools are refused in scheduled runs, so a background task can never pop a browser open

This is a **controlled WebView, not a hardened sandbox**. Avoid signing in or paying on pages opened here.

7. Verification status

Verified on a physical phone: address-bar search, page menu, agent-opened pages, reader view, bookmarks and history, and all AI actions. Present in the menu but not individually exercised on the device: find-in-page match counter, desktop-site toggle, text size and share. See the Test Report.

8. Requirements covered

FR-09 (web automation), FR-21 (browser tools for people), FR-22 (AI on web pages), NFR-02 and NFR-11 (security and privacy). Tests: `BrowserUrlTest`, `ControlledWebScraperTest`, and `AgentReliabilityTest.backgroundRunNeverUsesScreenBoundTools`.

User Guide

Talking to Aurviz

Type in the message box or tap the microphone. While Aurviz works, the header shows what it is doing ("Searching your inbox…"). Every tool it uses appears as a card, so you can see exactly what happened.

Examples:

- "Summarize my unread emails from today"
- "Draft a reply to the interview invitation, but don't send it"
- "Remember that my manager is Alex and her email is alex@company.com"
- "Every weekday morning at 8, summarize my inbox" (weekdays)
- "Every Monday and Thursday at 6 PM, check my emails" (specific days)
- "Weekends at 10 AM, summarize my newsletters"
- Advanced: a cron expression such as `0 9 * * 1-5` (minute hour day-of-month month day-of-week)
- "Open the BBC News website and tell me the top story"

Mailboxes

Choose where Aurviz reads and writes mail in **Settings, Mailbox**:

- **Test Gmail account** (default): a real Gmail account made for testing. The email, password and sign-in steps are shown in Settings. It can only send to itself or example.com addresses.
- **My own Gmail**: sign in with Google.
- **Demo mailbox**: sample emails on your phone. Works offline.

For real accounts, open Gmail itself to see the mail. For the demo mailbox, open the **inbox icon** in the chat header (or Settings, View demo mailbox) to see **Inbox, Sent and Drafts**. Tap a message to read it or have Aurviz summarize it. The lists update as soon as Aurviz drafts or sends, and the menu has **Reset demo mailbox**.

Email

- **Search and read:** Aurviz uses Gmail search syntax internally (`from:` , `subject:` , `is:unread` , `newer_than:3d`).
- **Drafts:** "draft…" or "don't send" always saves a draft.
- **Sending:** Aurviz shows **To, Subject and Body** in an editable dialog. Nothing is sent until you tap **Send email**. Scheduled tasks can never send.
- **Your own account:** Settings, Mailbox, My own Gmail, Connect Gmail. Sign-in happens on Google's page.

Memory

Aurviz remembers facts and preferences and uses them in later answers. The **Memory** tab lets you search, add, edit and delete. Deleting asks for confirmation. You can also say "forget my…".

Tasks

The **Tasks** tab lists routines. For each task you can **Run now**, **pause or resume** (daily tasks), **edit** or **delete**. The **History** tab keeps every run with the time taken. Tap a task's "View last result" to read what happened. If something failed you get a plain explanation and what to do next, with technical details one tap away.

Repeats can be daily, weekdays, weekends, specific days or a cron expression; the card shows it in words (for example "Weekdays at 09:00") and the next run time. Android may start a scheduled task a few minutes late to save battery.

Browser

Tap the globe in the chat header.

- Type a website or just words to search.
- Menu (three dots): Find in page, Reader view, Desktop site, Larger or smaller text, Share link, Copy link, Open in Chrome.
- Bottom bar: back, forward, home, bookmark star, bookmarks and history.
- **AI chips:** Summarize, Key facts, Explain simply, Ask about this page. The page content is sent to Aurviz only when you tap one of these.

Settings

- **Appearance:** System, Light or Dark.
- **Mailbox:** shared test inbox, demo mailbox or your own Gmail, plus View mailbox.
- **AI model:** your own Gemini key (optional if the build includes one).

- **Preferences:** default recipient and spoken replies.
- **Permissions:** shortcut to Android's permission page.
- **Performance:** measured response times.

Shortcuts

Long-press the Aurviz icon for Voice command, Summarize inbox and Tasks. Share text from another app and choose Aurviz to get a summary.

Troubleshooting

Problem	What to do
"Could not reach Aurviz"	The hosted server may be waking up. Wait a few seconds and tap Retry
"Gmail inbox access was not granted"	Settings, Mailbox, My own Gmail, Disconnect, then Connect Gmail again and approve all permissions
"Could not reach Aurviz" when signing in	The server may be waking up. Wait a few seconds and Retry
"The test account can only send to itself…"	Expected. Use the demo mailbox or your own Gmail to email other people
Google asks to verify it's you	Use the Demo mailbox
"Gemini API key is not configured"	Add a key in Settings, AI model
Task ran late	Normal Android battery behaviour; use Run now for immediate execution

Backend API

Base URL is the deployed HTTPS service. All `/api/v1` calls except the OAuth exchange need `Authorization: Bearer <session token>`. Errors return `{ "error": "message" }`.

Health

Method	Path	Notes
GET	`/health`	`{"status":"ok"}` when the database and job queue are reachable

Authentication

Method	Path	Description
GET	`/auth/google`	Starts Google OAuth (adds CSRF state), redirects to Google
GET	`/auth/google/callback`	Exchanges the code, stores the encrypted refresh token, redirects to `aurviz://oauth/callback?code=<one-time code>`
POST	`/api/v1/auth/exchange`	Body `{ "code": "<one-time code>" }`, returns `{ accessToken, expiresAt }`
GET	`/api/v1/auth/me`	Connected account and expiry
POST	`/api/v1/auth/session/refresh`	Rotates the session token
DELETE	`/api/v1/auth/session`	Revokes this session
DELETE	`/api/v1/auth/google`	Disconnects the account, cancels pending sends, revokes the Google token

Mailbox

Method	Path	Description
GET	`/api/v1/gmail/messages?q=&max=`	Search with Gmail syntax. `max` 1 to 15. Returns id, from, to, subject, date, snippet, unread
GET	`/api/v1/gmail/messages/:id`	One message as plain text (HTML converted, attachments skipped, 6,000 character cap)
POST	`/api/v1/gmail/drafts`	Body `{ recipient, subject, body }`. Saves a draft, sends nothing

Example:

```
GET /api/v1/gmail/messages?q=subject:interview%20newer_than:7d&max=5
Authorization: Bearer <token>
```

```
{ "query": "subject:interview newer_than:7d", "messages": [
  { "id": "19a1d…", "from": "HR <hr@example.com>", "subject": "Interview invitation",
    "date": "Thu, 12 Nov 2026", "snippet": "Please confirm…", "unread": true } ] }
```

Sending and scheduling

Method	Path	Description
POST	`/api/v1/emails/send`	Queue an immediate email. Header `Idempotency-Key` (8 to 120 safe characters) required. Returns 202
POST	`/api/v1/emails/scheduled`	Schedule one email for a future time (at least 30 s ahead)
GET	`/api/v1/emails/scheduled`	List scheduled emails with recent history
GET	`/api/v1/emails/scheduled/:id`	One email with execution history
PATCH	`/api/v1/emails/scheduled/:id`	Update a pending email (stale queued jobs cannot send the old version)
DELETE	`/api/v1/emails/scheduled/:id`	Cancel a pending email

Test account restrictions

The reviewer test account (`SHARED_TEST_ACCOUNT_EMAIL`) has its credentials shown in the app, so the server enforces:

- `POST /emails/send` and `POST /emails/scheduled` return **403** unless the recipient is the test account itself or an `example.com`, `example.org`, `example.net`, `*.example` or `*.test` address.
- `DELETE /auth/google` returns **403** (the grant cannot be revoked from the app). `DELETE /auth/session` ends only that device's session.

Delivery semantics

Gmail acceptance and the database commit cannot be atomic. Definite 4xx failures become `FAILED`; quota (429) is retried with backoff; network or 5xx outcomes become `UNKNOWN` and are **never** resent automatically. The same idempotency key never produces two messages.

Security

HTTPS, CSRF state on OAuth, one-time codes, hashed session tokens, AES-256-GCM refresh tokens, Helmet, rate limiting (120 requests per minute), 32 KB body limit, schema validation, header-injection rejection, redacted logs.

Build and Deployment

1. Build the Android app

Requirements: Android Studio, JDK 17, Android SDK 34.

```
./gradlew app:testDebugUnitTest      # run the tests
./gradlew app:assembleDebug          # APK at app/build/outputs/apk/debug/app-debug.apk
```

Bundle a Gemini key for evaluators (optional, recommended)

Create `local.properties` (it is git-ignored) and add:

```
sdk.dir=/path/to/Android/sdk
gemini.api.key=YOUR_GEMINI_KEY
```

Build again with `./gradlew clean app:assembleDebug` (use `clean` after changing `local.properties`: Kotlin inlines build-time constants, so an incremental build can keep the old value). The APK then works with no setup. A user's own key entered in Settings always takes priority. **Never commit `local.properties`.** A key inside an APK can be extracted, so use a free-tier key you can rotate, and restrict it in Google AI Studio.

The API base URL is `API_BASE_URL` in `app/build.gradle.kts`.

2. Backend (Node.js 22.12+, PostgreSQL)

```
cd backend
cp .env.example .env        # fill in the values below
npm ci
npx prisma migrate deploy
npm start
```

Variable	Meaning
`DATABASE_URL`	PostgreSQL connection string
`SESSION_SECRET`	32+ random bytes for the OAuth session cookie
`TOKEN_ENCRYPTION_KEY_B64`	base64 of 32 random bytes, encrypts refresh tokens
`GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`	Web OAuth client
`GOOGLE_REDIRECT_URI`	`https://<host>/auth/google/callback`
`ANDROID_REDIRECT_URI`	`aurviz://oauth/callback`
`PUBLIC_BASE_URL`	Public HTTPS URL of the service
`ROLE`	`all`, `api` or `worker`
`SHARED_TEST_ACCOUNT_EMAIL`	The throwaway account reviewers use (default `adityatesting404@gmail.com`)

Generate secrets with `openssl rand -base64 32`.

3. Google Cloud setup (for real Gmail)

1. Create a project and enable the **Gmail API**.
2. OAuth consent screen: add the scopes `gmail.send`, `gmail.readonly`, `gmail.compose` (plus `openid`, `email`).
3. Add test users (or publish unverified, see the Evaluator Guide).
4. Create an OAuth client of type **Web application** and add the exact HTTPS callback URL.
5. Put the client id and secret in the backend environment (never in the app or the repository).

`gmail.readonly` is a restricted scope: it works for listed test users, and a public release needs Google verification. Accounts connected before the read scopes were added must disconnect and reconnect.

4. Preparing the reviewer test account

1. Create a throwaway Gmail account (no private data) and add it as an OAuth test user.
2. Put its credentials in your git-ignored `local.properties` (`test.account.email`, `test.account.password`) so they are shown inside the APK. They are never committed.
3. Reviewers choose **Test Gmail account**, follow the on-screen steps and sign in on Google's page. Each sign-in is a normal OAuth connection to the backend.
4. **Publish the OAuth consent screen "In production"** (unverified is fine), otherwise sign-ins by accounts that are not listed as test users are blocked and test-user tokens expire after 7 days. Reviewers will see the "Google hasn't verified this app" warning and click Continue.

5. Google may add a "verify it's you" challenge to a shared account used from many devices. The demo mailbox is the fallback.

Only use a throwaway account. The server restricts it to sending to itself or example.com addresses and refuses to disconnect it.

5. Hosting notes

- **Pooled databases (for example Neon's `-pooler` URL):** `prisma migrate deploy` can time out waiting for a Postgres advisory lock (error P1002). The `prisma:migrate` script therefore sets `PRISMA_SCHEMA_DISABLE_ADVISORY_LOCK=1`. A direct (non-pooled) connection string is also fine.
- Any host with PostgreSQL works. The reference deployment is a free web service plus a free database, which sleeps when idle. The app wakes it at launch and allows a 45 s first request.
- Run at least one worker (`ROLE=all` does both API and worker).
- A push to GitHub only updates the running service if auto-deploy is enabled. Otherwise use **Manual Deploy, Deploy latest commit** in the hosting dashboard. New endpoints return 404 until the new version is deployed.
- Enable database TLS and backups for production.

6. Continuous integration

`.github/workflows/build.yml` runs the Android unit tests and builds the APK, and runs the backend tests and `npm audit` on every push.